

บทที่ 4

การออกแบบและพัฒนาระบบ

(ส่วนการแสดงผล)

4.1 การออกแบบตารางฐานข้อมูล

ฐานข้อมูลที่ใช้ในระบบของส่วนการแสดงผลนี้ใช้ MySQL เป็นฐานข้อมูล ซึ่งประกอบไปด้วยตารางทั้งหมด 7 ตารางคือ

4.1.1 ตาราง Member

เป็นตารางที่เก็บรายละเอียดของสมาชิกของระบบ โดยมีฟิลด์ต่างๆ ดังนี้คือ

ฟิลด์	รายละเอียด
id	หมายเลขลำดับสมาชิก
usr	เป็นชื่อที่ใช้ในการล็อกอิน (login) เข้าสู่ระบบสมาชิก
pass	รหัสที่ใช้การเข้ารับบริการของระบบ
name	ชื่อและนามสกุลของสมาชิก
email	อีเมลที่ใช้ในการติดต่อ
district	เขตพื้นที่ของกรุงเทพมหานครฯ เช่น ลาดกระบัง, พระโขนง
address	ที่อยู่ของสมาชิก
mobile	หมายเลขโทรศัพท์เคลื่อนที่ของสมาชิก
Provider	ผู้ให้บริการโทรศัพท์เคลื่อนที่
Mo_usr	ชื่อที่ใช้ในการล็อกอินของโทรศัพท์เพื่อส่ง SMS
Mo_pass	รหัสผ่านที่ใช้เพื่อส่ง SMS
Start1	จุดเริ่มต้นที่ 1 ที่สมาชิกต้องการให้ระบบค้นหาเส้นทางให้
dest1	จุดสิ้นสุดที่ 1 ที่สมาชิกต้องการให้ระบบค้นหาเส้นทางให้
Start2	จุดเริ่มต้นที่ 2 ที่สมาชิกต้องการให้ระบบค้นหาเส้นทางให้
Dest2	จุดเริ่มต้นที่ 2 ที่สมาชิกต้องการให้ระบบค้นหาเส้นทางให้

ตารางที่ 4-1 แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง Member

4.1.2 ตาราง Matching

เป็นตารางที่ใช้ทำการเปรียบเทียบเพื่อแปลงจากชื่อบริเวณต่างๆ ซึ่งเป็นได้ทั้งชื่อถนน, สถานที่ และชื่อแยก ให้กลายเป็นชื่อแยกทั้งหมดซึ่งก็คือจุดที่สามารถเกิดการเลี้ยวเปลี่ยนเส้นทางการเดินทางได้ เช่น บริเวณยูเทิร์น (U-turn) ของถนน และสี่แยก

เนื่องจากว่าในการทำงานของส่วนที่เป็นระบบค้นหาเส้นทางอัจฉริยะส่วนประมวลผลนั้น จะเป็นการคำนวณจากโหนด (node) ดังกล่าวทั้งสิ้น ดังนั้นเพื่อความยืดหยุ่นในการใช้งานของผู้บริการ ให้มีความหลากหลายจึงต้องทำการแปลงก่อน แล้วก็ส่งต่อให้ส่วนประมวลผลทำงานต่อไป

ฟิลด์	รายละเอียด
word	ชื่อบริเวณซึ่งเป็นได้ทั้งชื่อถนน, สถานที่และชื่อแยก
inter1	เป็นชื่อที่ใช้ในการล็อกอิน (login) เข้าสู่ระบบสมาชิก
inter2	รหัสที่ใช้การเข้ารับบริการของระบบ

ตารางที่ 4-2 แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง Matching

4.1.3 ตาราง Report

เป็นตารางที่ใช้ในการรายงานอุบัติเหตุหรือเกิดสิ่งกีดขวางการจราจรที่เกิดขึ้น โดยการค้นหาจะ เริ่มจากชื่อถนนหรือชื่อเขตของกรุงเทพมหานคร

ฟิลด์	รายละเอียด
street	ชื่อถนนที่เกิดอุบัติเหตุหรือเกิดสิ่งกีดขวางการจราจร
district	ชื่อเขตที่เกิดอุบัติเหตุหรือเกิดสิ่งกีดขวางการจราจร
comment	รายงานอุบัติเหตุ หรือเกิดสิ่งกีดขวางการจราจร
date	วันที่ที่เกิดรายงานนั้นๆ

ตารางที่ 4-3 แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง Report

4.1.4 ตาราง District

เก็บรายชื่อของเขตต่างๆภายในกรุงเทพมหานครฯ เพื่อการเปลี่ยนแปลงในภายหลัง เช่น การเพิ่มหรือลบเขต จะสามารถทำได้ง่ายขึ้นโดยกระทำที่จุดเดียว

ฟิลด์	รายละเอียด
id	หมายเลขลำดับของเขตภายในกรุงเทพมหานครฯโดยเรียงตามตัวอักษร
distr_name	รายชื่อเขตของกรุงเทพมหานคร

ตารางที่ 4-4 แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง Report

4.1.5 ตาราง Board_quest

ใช้เก็บรายละเอียดของเว็บบอร์ด ซึ่งตารางนี้จะเก็บเกี่ยวกับกระทู้หลัก (หัวข้อกระทู้) ทั้งหมดที่มีของเว็บบอร์ด แต่จะไม่ได้ทำการเก็บเกี่ยวกับรายละเอียดของการตอบกระทู้

ฟิลด์	รายละเอียด
id	หมายเลขลำดับของหัวข้อกระทู้ โดยเรียงตามเวลาที่สร้างก่อนหลัง
topic	ชื่อหัวข้อกระทู้
name	ชื่อผู้ตั้งกระทู้ (ชื่อที่ใช้ในการล็อกอินของสมาชิก)
email	อีเมลของผู้ตั้งกระทู้ที่ได้ให้ไว้ในฐานข้อมูลสมาชิก
detail	รายละเอียดของหัวข้อกระทู้
ip	หมายเลขไอพีแอดเดรส (IP address) ของเครื่องที่ทำการสร้างกระทู้
datepost	วันที่และเวลาที่สร้างกระทู้
lastpost	วันที่และเวลาครั้งล่าสุดที่ได้มีการเปลี่ยนแปลงกระทู้ (ตอบหรือสร้าง)
Lastname	ชื่อล็อกอินของผู้ที่ทำการเปลี่ยนแปลงครั้งล่าสุด
View	จำนวนของผู้ที่เข้ามาดูกระทู้นี้
ans	จำนวนผู้ที่ทำการตอบกระทู้

ตารางที่ 4-5 แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง Board_quest

4.1.6 ตาราง Board_ans

ใช้เก็บรายละเอียดของเว็บบอร์ด ซึ่งตารางนี้จะเก็บเกี่ยวกับการโพสต์ตอบกระทู้ โดยเชื่อมโยงกับตาราง Board_quest โดยใช้หมายเลข id_quest ของตารางนี้ให้ตรงกับหมายเลข id ในตาราง Board_quest หากตรงกันหมายถึงคำตอบกระทู้นี้เป็นของกระทู้นั้นๆ

ฟิลด์	รายละเอียด
id_ans	หมายเลขของคำตอบกระทู้
id_quest	หมายเลขของกระทู้ซึ่งตรงกันกับของตาราง Board_quest
name	ชื่อล็อกอินของผู้ตอบกระทู้
email	อีเมลของผู้ตอบกระทู้ที่ได้ให้ไว้ในฐานข้อมูล
ip	หมายเลขไอพีแอดเดรส (IP address) ของเครื่องที่ทำการตอบกระทู้
datepost	วันที่และเวลาที่ได้ทำการตอบกระทู้

ตารางที่ 4-6 แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง Board_quest

4.1.7 ตาราง Mapinfo

เป็นตารางที่จะถูกเรียกใช้งานโดย Applet เพื่อใช้ในการแสดงแผนที่ และสร้างจุดที่แสดงให้เห็นถึงบริเวณที่มีโหนด (จุดที่สามารถเกิดการเคลื่อนเพื่อเปลี่ยนเส้นทางการเดินทางได้)

โดยที่ applet จะนำค่า x และ y จากฐานข้อมูลมาทำการวาดโหนด ถ้า applet ทำการแสดงผลภาพใดอยู่ก็จะทำการวาดโหนดที่มีฟิลด์ image ตรงกับชื่อรูปนั้นๆ

ฟิลด์	รายละเอียด
id	หมายเลขของโหนด
node	ชื่อโหนด
x_coord	ค่าตามแกน x ของโหนดเรียงจากซ้ายไปขวาโดยเริ่มที่ 0 มีค่าถึง 600
y_coord	ค่าตามแกน y ของโหนดเรียงจากบนลงล่างโดยเริ่มที่ 0 มีค่าถึง 600
junction	จำนวนแยกเส้นทางที่สามารถไปได้ของโหนด (ยูเทิร์นมีค่าเป็น 0)
image	ชื่อรูปภาพของแผนที่ที่จะทำการเรียกใน applet
default_district	เป็นชื่อเขตที่ใช้ในการบ่งบอกให้ทำการแสดงแผนที่ของเขตนั่นๆ

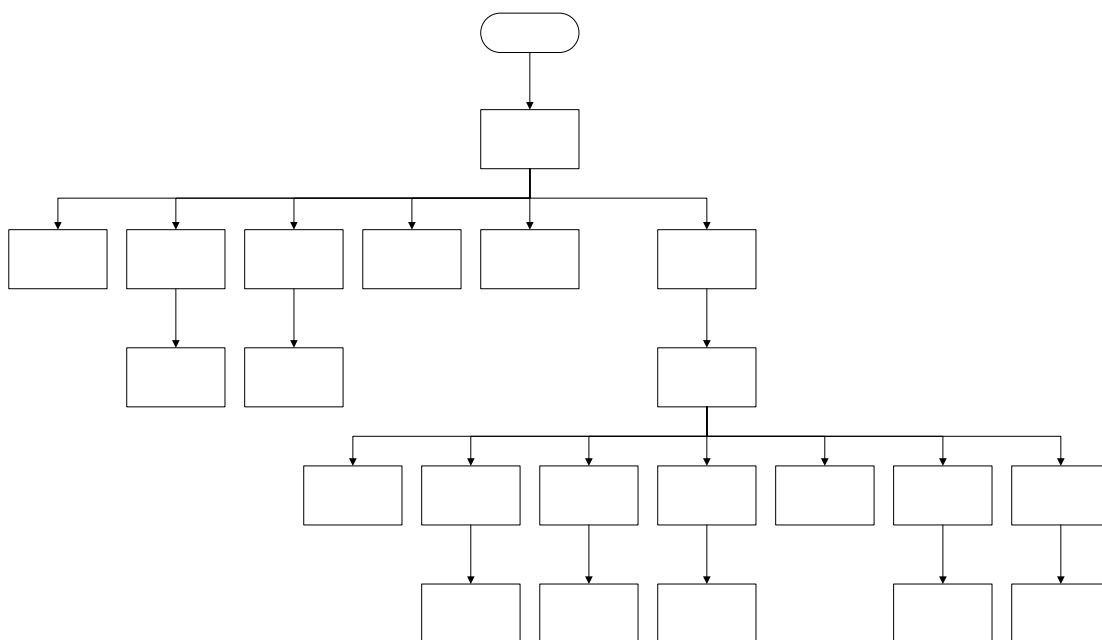
ตารางที่ 4-7 แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง *Board_quest*

4.2 ขั้นตอนการทำงานส่วนแสดงผลของเว็บไซต์

การทำงานของส่วนเว็บไซต์จะอยู่บนเครื่องคอมพิวเตอร์ โดยใช้สคริปต์ JSP ในการเขียน โดยใช้ Tomcat เป็นเว็บเซิร์ฟเวอร์ และติดต่อกับฐานข้อมูลซึ่งก็คือ MySQL โดยผ่านทาง JDBC โดยไดรเวอร์ (Driver) ที่ชื่อว่า “org.gjt.mm.mysql.Driver” ซึ่งเครื่องที่เป็นเว็บเซิร์ฟเวอร์และเซิร์ฟเวอร์ฐานข้อมูลเป็นเครื่องเดียวกัน ลักษณะการทำงานจะแบ่งเป็นแต่ละเพจดังนี้ คือ

1. หน้าแรกหรือโฮมเพจ (home.jsp)
2. หน้าสมัครสมาชิกใหม่ (register.jsp)
3. หน้าแก้ไขข้อมูลส่วนตัวของสมาชิก (edit.jsp และ edit_cont.jsp)
4. หน้าล็อกอินหรือหน้าเข้าสู่ระบบสมาชิก (login.jsp)
5. หน้าล็อกเอาต์ออกจากระบบ (logout.jsp)
6. หน้าแสดงแผนที่และรายละเอียดของเส้นทางจราจร (map.jsp และ member_map.jsp)
7. หน้าค้นหาเส้นทางจราจร (routing.jsp และ member_routing.jsp)
8. หน้าแสดงรายงานอุบัติเหตุและสิ่งกีดขวางการจราจร (report.jsp และ member_report.jsp)
9. หน้าแสดงรายงานสถิติวิทยุออนไลน์ จส.100 (radio.jsp และ member_radio.jsp)
10. หน้าต้อนรับเข้าสู่ระบบสมาชิกหลังทำการล็อกอิน (member_section.jsp)
11. หน้าแสดงรายชื่อกระทู้ทั้งหมดของเว็บบอร์ด (webboard.jsp)
12. หน้าสร้างกระทู้ใหม่หรือหน้าโพสต์ (addnew.jsp)
13. หน้าแสดงและตอบกระทู้ (view.jsp)
14. หน้าแสดงความผิดพลาดของเซิร์ฟเวอร์ (error.jsp)
15. ไฟล์เก็บส่วนหัวและแถบเมนูด้านข้าง (header.jsp, member_header.jsp และ member_left.jsp)
16. โปรแกรม Java Applet ที่ใช้แสดงแผนที่ (Map.class)

หน้าที่ขึ้นต้นด้วย “member” ทั้งหมดและหน้าที่เกี่ยวกับเว็บบอร์ดจะมีตรวจสอบก่อนการเข้ามาดูทุกครั้งว่าบุคคลนั้นๆได้ผ่านการล็อกอินมาอย่างถูกต้องแล้วหรือยัง และจะมีการเรียกใช้งาน member_left.jsp และ member_header.jsp ซึ่งเป็นส่วนเมนูที่อยู่ทางด้านซ้ายและด้านบนของเว็บเพจตามลำดับ ส่วนหน้าเว็บเพจอื่นๆจะเรียกใช้ header.jsp ทุกๆเพจ ดังนั้นการที่จะแก้หน้าตาของเว็บเพจจึงสามารถกระทำได้ที่เดียว



รูปที่ 4-1 แสดงลำดับการใช้งานของเว็บเพจ

และเมื่อเกิดข้อผิดพลาดขึ้น สำหรับทุกๆเพจจะเรียกไปยัง error.jsp

4.2.1 หน้าแรกหรือโฮมเพจ (home.jsp)

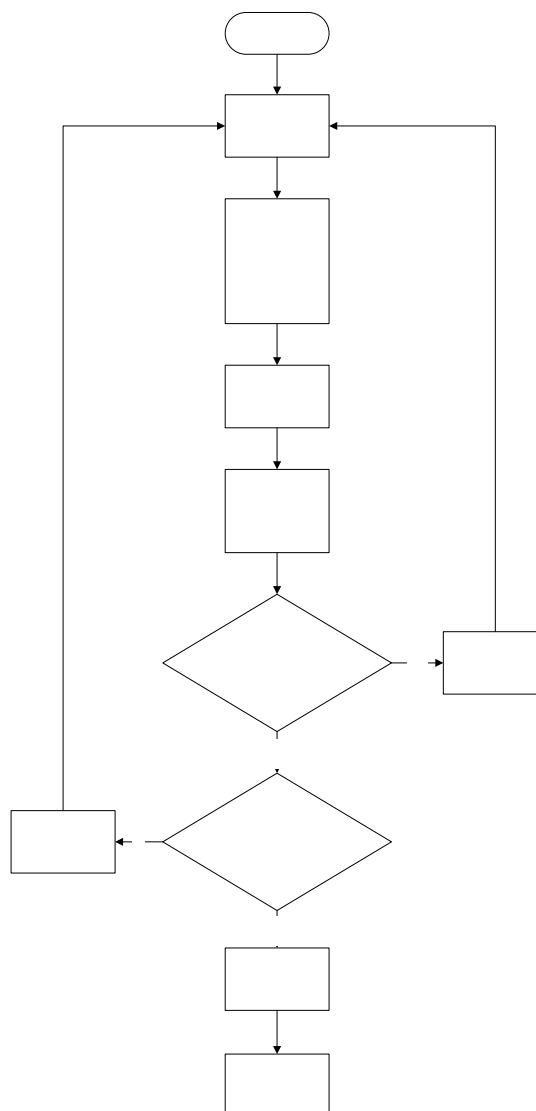
มีการแสดงลิงค์ไปยังเพจต่างๆภายในเว็บไซต์ และยังรวบรวมลิงค์ที่เกี่ยวข้องกับการจราจรภายในเขตกรุงเทพมหานครด้วย



รูปที่ 4-2 แสดงหน้าตาของ home.jsp

4.2.2 หน้าสมัครสมาชิกใหม่ (register.jsp)

เป็นที่ให้สมาชิกทำการกรอกข้อมูลเพื่อนำข้อมูลไปลงในฐานข้อมูลในตารางที่ชื่อ member

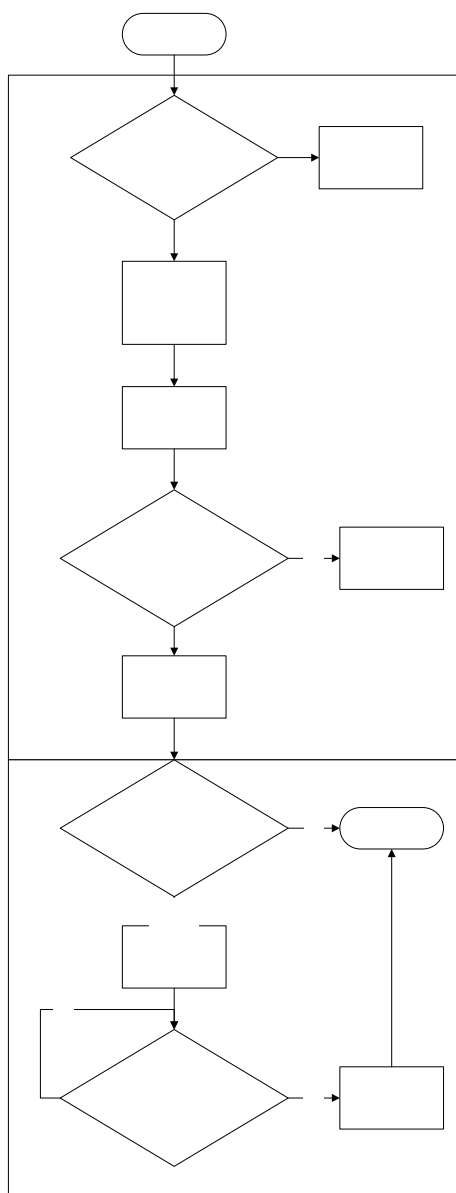


รูปที่ 4-3 แสดงโฟลว์ชาร์ตของ register.jsp

ในการตรวจสอบความถูกต้องของอินพุตนั้นจะตรวจสอบว่า ชื่อ, อีเมล, รหัสผ่านทั้ง 2 ครั้ง และ ชื่อล็อกอินต้องอยู่ในรูปแบบที่ถูกต้อง คือ ต้องมีการป้อนและไม่ประกอบด้วยอักขระพิเศษ รวมทั้งตรวจสอบว่าจุด start และ destination ที่เป็นคู่เดียวกันนั้นต้องไม่ใช่จุดเดียวกัน และเมื่อมีการกรอกอินพุตของคู่ใดแล้ว อินพุตคู่อื่นๆจะต้องมีการกรอกด้วย

4.2.3 หน้าแก้ไขข้อมูลส่วนตัวของสมาชิก (edit.jsp และ edit_cont.jsp)

เป็นหน้าสำหรับสมาชิกเท่านั้น ไว้ทำการแก้ไขข้อมูลส่วนตัวของตัวเองที่เคยให้ไว้ในฐานข้อมูล
จากขั้นตอนการสมัครสมาชิก



รูปที่ 4-4 แสดงโฟลว์ชาร์ตของ *edit.jsp* และ *edit_cont.jsp*

สำหรับการตรวจสอบอินพุตทุกอย่างจะเหมือนกันกับ register.jsp

4.2.4 หน้าล็อกอินหรือหน้าเข้าสู่ระบบสมาชิก (login.jsp)

สำหรับให้สมาชิกทำการล็อกอินเพื่อเข้าสู่ระบบ โดยการใชัพิสูจน์ตน (Authentication) แบบ
 username,password ซึ่งเมื่อสมาชิกผ่านการล็อกอินแล้วจะมีความสามารถในการเข้าสู่เว็บบอร์ดได้ และ
 ในการใช้งานระบบจะเพิ่มความสะดวกสบายมากขึ้นโดยการใช้ข้อมูลของสมาชิกเองในฐานะข้อมูลที่เคย
 ใช่วิกับระบบ

ซึ่งขั้นตอนนี้จะใช้เทคนิคของตัวแปรเซสชัน (session parameter) คือ จะใช้หมายเลขเซสชันของ http connection เป็นเหมือนกับหมายเลขตัวแทนในการติดต่อกับเซิร์ฟเวอร์ แล้วเก็บข้อมูลในฝั่งเซิร์ฟเวอร์สำหรับผู้ผ่านการล็อกอินอย่างถูกต้องโดยใช้หมายเลขเซสชันเป็นตัวบ่งบอกว่าต้องเก็บที่ตัวแปรใด

ดังนั้นในการตรวจสอบสิทธิของสมาชิกจะใช้ตัวแปรเซสชันที่เก็บในฝั่งเซิร์ฟเวอร์เป็นตัวตรวจสอบ (ผู้ผ่านการล็อกอินจะมีตัวแปร ผู้ที่ไม่ผ่านจะไม่มี)

ซึ่งโค้ดส่วนนี้จะเป็นดังนี้

```
if (session.getAttribute("auth") == null)
{
    response.sendRedirect("login.jsp");
    return;
}
```

4.2.5 หน้าล็อกเอาต์ออกจากระบบ (logout.jsp)

เป็นการลบตัวแปรเซสชันออกจากฝั่งเซิร์ฟเวอร์ทั้งหมด โดยใช้หมายเลขเซสชันเป็นตัวบ่งบอกเช่นเดียวกันกับขั้นตอนล็อกอิน

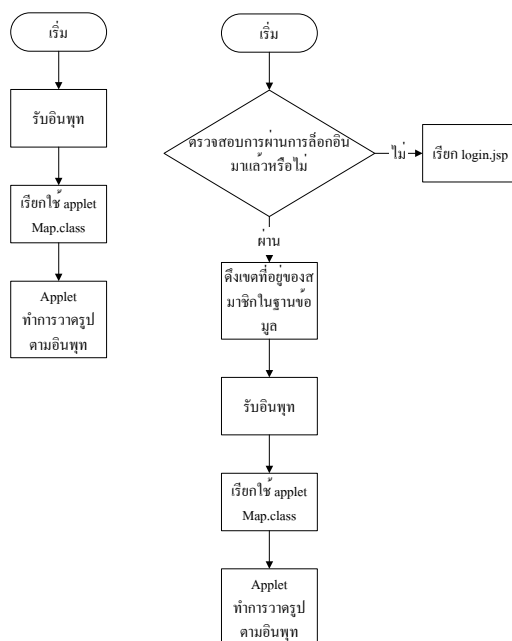
และสำหรับการที่สมาชิกที่ผ่านการล็อกอินแล้ว แต่ไม่ได้ทำการติดต่อกับระบบเป็นเวลานาน (20 นาที) จะทำให้เซสชันนั้นปิดตัวลงไป จะเสมือนกับว่าสมาชิกได้ทำการล็อกเอาต์ออกไปแล้ว

ซึ่งส่วนนี้จะใช้คำสั่ง

```
<%
    session.invalidate();
    response.sendRedirect("home.jsp");
%>
```

4.2.6 หน้าแสดงแผนที่และรายละเอียดของเส้นทางจราจร (map.jsp และ member_map.jsp)

จะแสดงแผนที่พร้อมด้วยสภาพการจราจรให้กับผู้ใช้ดู โดยต้องให้ผู้ใช้ทำการเลือกเขตที่ต้องการจะดูก่อน แต่สำหรับสมาชิกจะแสดงแผนที่ของเขตที่ตรงกับในฟิลด์ district ในตาราง member ในฐานข้อมูลเลย โดยแผนที่นั้นจะใช้ applet เป็นตัววาด (Map.class) และสำหรับ applet เพื่อนำเมาส์ไปวางไว้บนตำแหน่งที่เป็นโหนด ก็จะแสดงรายละเอียดของโหนดนั้นๆออกมา



รูปที่ 4-5 แสดงโฟลว์ชาร์ตของ map.jsp และ member_map.jsp

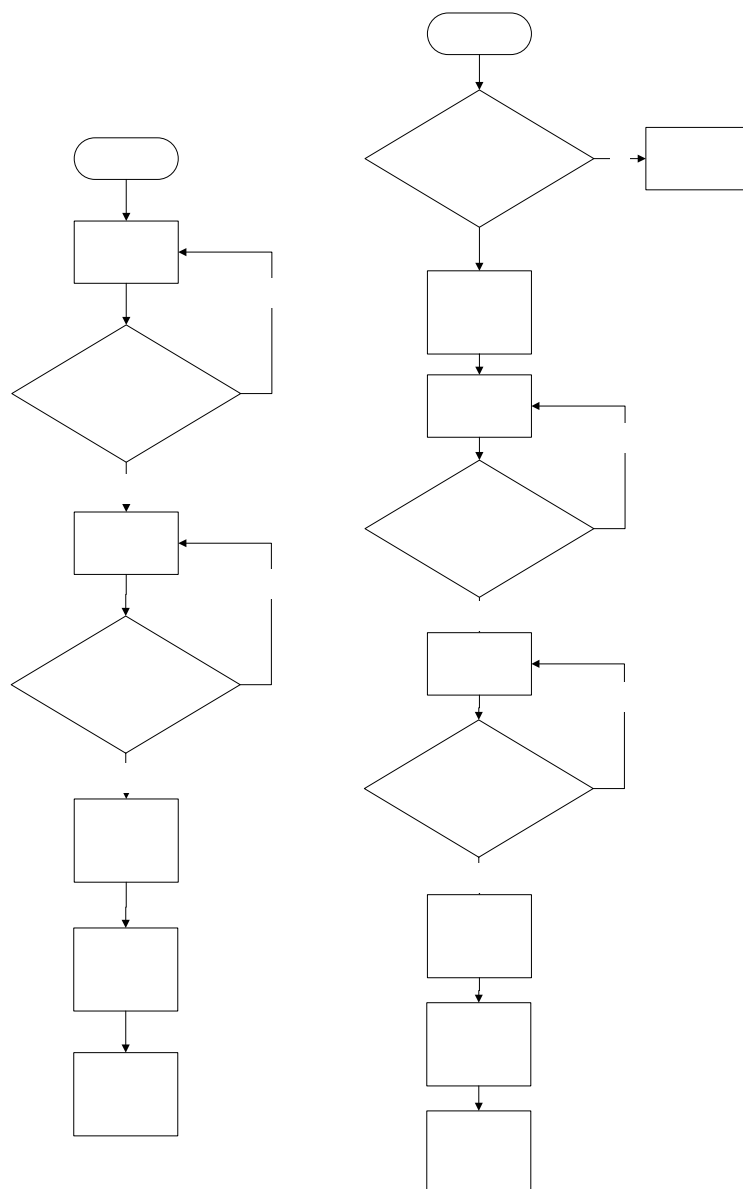
4.2.7 หน้าค้นหาเส้นทางการจราจร (routing.jsp และ member_routing.jsp)

ให้ผู้ใช้ได้ทำการค้นหาบริเวณที่ต้องการก่อน โดยประเภทของบริเวณจะเป็นได้คือ ชื่อถนน, ชื่อโหนดและสถานที่ เช่น ถ้าทำการค้นหาคำว่า “ราช” ผลที่ได้จากการค้นหาก็คือ ราชประสงค์ ราชดำริ ราชดำเนิน เป็นต้น และผลการค้นหาจะแสดงออกมาเป็น radio box ให้ทำการเลือกว่าจะเลือกจากจุดเริ่มต้นใดไปยังจุดสิ้นสุดใด แต่สำหรับสมาชิกจะแสดงจุดเริ่มต้นและสิ้นสุดที่มีอยู่ในฐานข้อมูลเลยโดยไม่ต้องผ่านขั้นตอนการค้นหา

ซึ่งขั้นตอนต่อไปก็จะทำการเปลี่ยนจากจุดหรือบริเวณที่รับเข้ามาเป็นโหนดทั้งหมด เช่น หากเลือกเป็นชื่อถนนจะได้โหนด 2 โหนดด้วยกัน คือ แยกที่ถนนนั้นๆสิ้นสุด โดยที่การเปลี่ยนจากจุดหรือบริเวณเป็นโหนดนั้นจะใช้ข้อมูลในตาราง matching ในฐานข้อมูล (เปลี่ยนจากฟิลด์ word ไปเป็นฟิลด์ inter1 และ inter2) ดังนั้นจะเห็นได้ว่าจากจุดหรือบริเวณ 2 จุด (คือต้นทางและปลายทาง) จะได้เป็นโหนดทั้งหมด 4 โหนด

การจับคู่โหนดนั้นก็จะจับคู่ได้ทั้งหมด 4 แบบด้วยกัน แต่สำหรับส่วนประมวลผลนั้นจะทำงานได้แค่ทีละคู่เท่านั้น ดังนั้นจึงต้องส่งให้แก่ส่วนประมวลผลทีละคู่ และจะได้ผลการค้นหา 4 ผลด้วยกัน จากนั้นจึงตรวจสอบว่าเส้นทางใดมีผลตรงกับความต้องการของผู้ใช้มากที่สุด จึงเลือกผลนั้นมาทำการแสดงเพียงผลเดียว

ผู้ใช้งานสามารถเลือกรูปแบบการค้นหาได้ทั้งสิ้น 3 รูปแบบ คือ ระยะทางที่สั้นที่สุด ค่าใช้จ่ายที่ใช้ในการเดินทางน้อยที่สุด และระยะเวลาในการเดินทางน้อยที่สุด โดยทั้ง 3 แบบนี้สามารถนำมาใช้รวมกันอย่างไรก็ได้



รูปที่ 4-6 แสดงโฟลว์ชาร์ตของ *routing.jsp* และ *member_routing.jsp*

สำหรับขั้นตอนการตรวจสอบอินพุตครั้งแรกนั้นจะดูว่าผู้ใช้ได้พิมพ์อินพุตเข้ามาหรือไม่ ถ้าพิมพ์เข้ามาก็จะตัดช่องว่างด้านหน้าและหลังของค่าที่รับเข้ามาก่อน แล้วค่อยนำมาค้นหาในฐานข้อมูล สำหรับการตรวจสอบอินพุตส่วนที่หลังนั้นจะดูว่าค่าจาก radio box ที่รับเข้ามา คือจุดเริ่มต้นและสิ้นสุดเป็นจุดเดียวกันหรือไม่ ถ้าเป็นจุดเดียวกันต้องทำการเลือกใหม่

4.2.8 หน้าแสดงรายงานอุบัติเหตุและสิ่งกีดขวางการจราจร (*report.jsp* และ *member_report.jsp*)

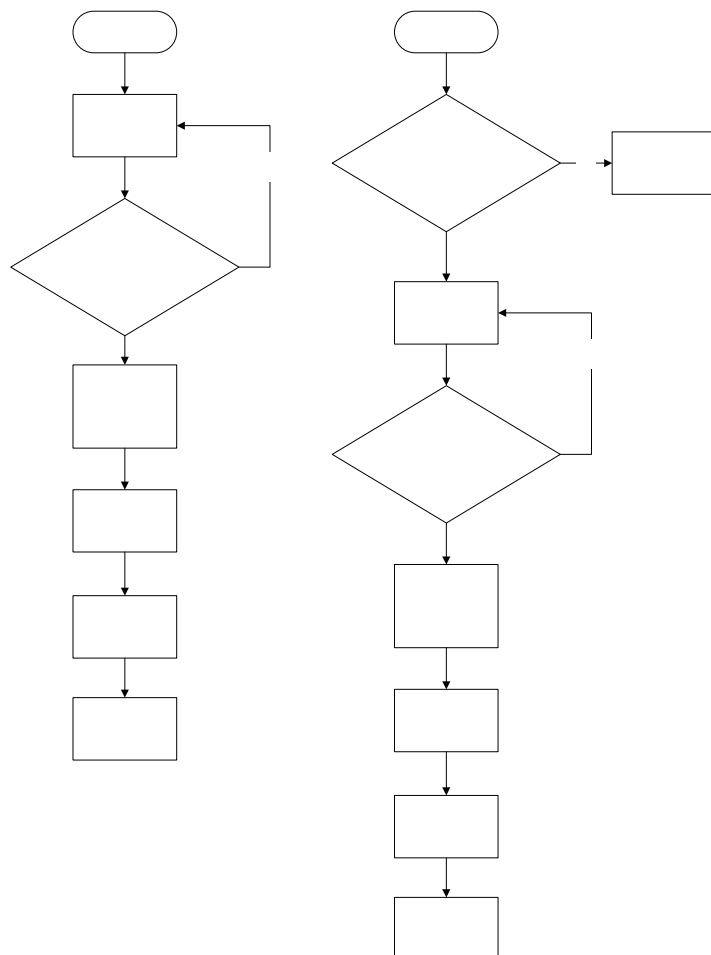
ให้ผู้ใช้ทำการค้นหาบริเวณที่ต้องการดูรายงานอุบัติเหตุหรือสิ่งกีดขวางการจราจรก่อน โดยบริเวณนั้นต้องเป็นได้ทั้งชื่อเขตและชื่อถนน คือถ้าเลือกค้นหาแบบชื่อเขต เมื่อทำการค้นหาแล้วผลที่จาก

ตรวจสอบ

อินพุต

การค้นหาคือเป็นรายชื่อถนนที่อยู่ในเขตนั้นๆและมีรายงานเข้ามาทั้งหมด แต่ถ้าเลือกเป็นชื่อถนน
ผลที่ได้จากการค้นหาคือเป็นเพียงถนนที่ค้นเจอเท่านั้น

จากนั้นผลของการค้นหาคือจะแสดงออกมาเป็น check box เพื่อให้เลือกแสดงรายงานเฉพาะบริเวณ
ที่ต้องการเท่านั้น



รูปที่ 4-7 แสดงโฟลว์ชาร์ตของ *report.jsp* และ *member_report.jsp*

4.2.9 หน้าแสดงรายงานสดวิทยุออนไลน์ จส.100 (radio.jsp และ member_radio.jsp)

เรียกใช้ plugin ของ Windows Media Player ใน Microsoft Internet Explorer มาทำการเล่น URL
ของสตรีมที่เป็นรายงานสดของวิทยุ จส.100

โดยมีคำสั่งดังนี้คือ

```

<Embed SRC="mms://live.virginradiothailand.com/105.5"
Width="337"
Height="70"
AutoStart="false"
TYPE="application/x-mplayer2"
  
```

```

pluginspage="http://www.microsoft.com/Windows/MediaPlayer/"
Name="objMediaPlayer"
autoSize="false"
showdisplay="false"
displaySize="0"
center="true"
ShowControls="true"
ShowStatusBar="true"
AnimationAtStart="false" border="2"></embed>

```



รูปที่ 4-8 แสดงplugin ของ Windows Media Player

4.2.10 หน้าต้อนรับเข้าสู่ระบบสมาชิกหลังทำการล็อกอิน (member_section.jsp)

เป็นหน้าต้อนรับสมาชิกผู้ที่ผ่านการล็อกอินเข้าสู่ระบบมาอย่างถูกต้องแล้วเท่านั้น

4.2.11 หน้าแสดงรายชื่อกระทู้ทั้งหมดของเว็บบอร์ด (webboard.jsp)

แสดงหัวข้อกระทู้ทั้งหมดที่มี โดยจะแบ่งแสดงเป็นหน้าๆ หน้าละ 10 หัวข้อกระทู้ ดังนั้นจึงต้องคำนวณก่อนว่าจะมีจำนวนหน้าทั้งหมดกี่หน้า

```
int total_page = (int)Math.ceil((double)total_row/(double)row_page);
```

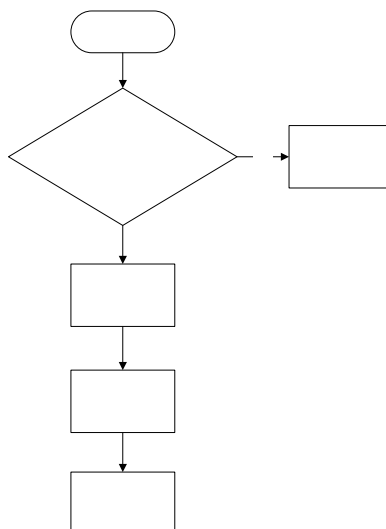
จากนั้นทำการคำนวณต่อว่า จะต้องดึงข้อมูลจากฐานข้อมูลเข้ามาตั้งแต่กระทู้ที่เท่าไรถึงเท่าไร โดยการหาจุดเริ่มต้นการ

```
start = (screen - 1)*row_page
```

screen คือ หมายเลขหน้าที่ต้องการเปิดดู จากนั้นก็ทำการดึงข้อมูลออกมาจำนวน 10 กระทู้

ทั้งนี้หมายเลขก่อนหลังของกระทู้จะต้องเรียงตามวันเวลาที่สร้างกระทู้คือ สร้างก่อนจะมี

หมายเลขต่ำกว่า



รูปที่ 4-9 แสดงโฟลว์ชาร์ตของ *webboard.jsp*

4.2.12 หน้าสร้างกระทู้ใหม่หรือหน้าโพสต์ (addnew.jsp)

สำหรับสมาชิกเพื่อทำการสร้างกระทู้ใหม่ โดยจะมีการเก็บวันเวลาที่โพสต์ หมายเลข IP address ของเครื่องที่ใช้ติดต่อเข้ามาด้วย ส่วนข้อมูลอื่นๆจะเก็บจากค่าในฐานข้อมูลของสมาชิกคนนั้นๆ เช่น อีเมลแอดเดรส

โดยที่เพจนี้จะมีการเรียกเมธอด (Method) ชื่อ `filter_topic` และ `filter_comment` ด้วย โดยที่ภายในเมธอดนี้จะมีรายละเอียดดังนี้

```

public static String filter_topic(String input)
{
    StringBuffer temp = new StringBuffer(input.length());
    char c;
    int j=1;
    for (int i=0;i<input.length();i++)
    {
        c = input.charAt(i);
        if (c == '<') temp.append("&lt;");
        else if (c == '>') temp.append("&gt;");
        else if (c == "\"") temp.append("&#039;");
        else if (c == "'") temp.append("&quot;");
        else if (c == '&') temp.append("&amp;");
        else if (c == '\\') temp.append("&#92;");
        else temp.append(c);
    }
}
  
```

```
        if (j == 30)
        {
            temp.append("<br>");
            j = 0;
        }
        j++;
    }
    return(temp.toString());
}
```

```
public static String filter_comment(String input)
{
    StringBuffer temp = new StringBuffer(input.length());
    char c;
    int j=1;
    for (int i=0;i<input.length();i++)
    {
        c = input.charAt(i);
        if (c == '<') temp.append("&lt;");
        else if (c == '>') temp.append("&gt;");
        else if (c == '\\') temp.append("&#039;");
        else if (c == '\"') temp.append("&quot;");
        else if (c == '&') temp.append("&amp;");
        else if (c == '\\') temp.append("&#92;");
        else temp.append(c);

        if (j == 50)
        {
            temp.append("<br>");
            j = 0;
        }
        if (c == '\\n')
            j = 0;
    }
}
```

```

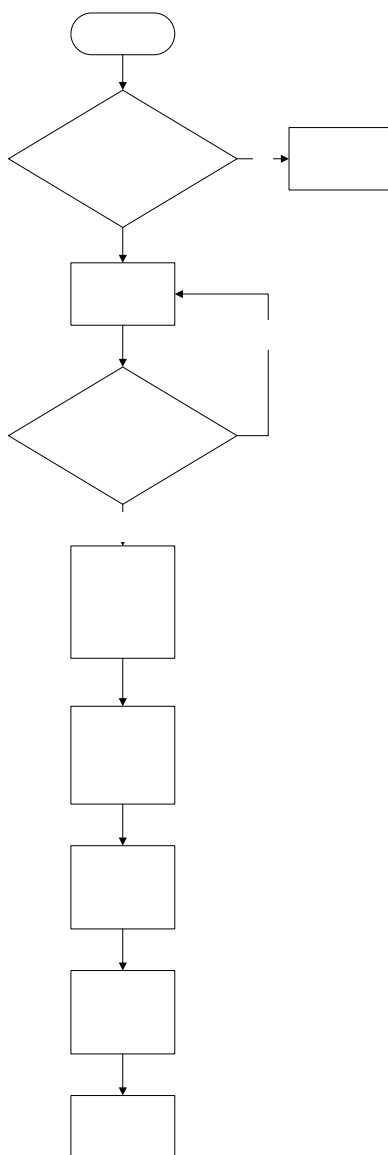
        j++;
    }
    return(temp.toString());
}

```

คือมีหน้าที่ในการแปลงข้อความที่รับมาจากสมาชิก แล้วทำการเปลี่ยนจากอักขระพิเศษที่จะทำให้เกิดปัญหาหรือเป็นอันตรายแก่เซิร์ฟเวอร์ได้เป็นอักขระอื่นๆดังนี้คือ

<	เป็น	<
>	เป็น	>
\	เป็น	'
"	เป็น	"
&	เป็น	&
\\	เป็น	\

และจะมีการตัดค่าความยาวของตัวอักษร คือ เมื่อข้อความมีความยาวเกิน 50 ตัวและ 30 ตัวอักษร สำหรับ filter_comment และ filter_topic ตามลำดับ โดยไม่มีการเว้นวรรคหรือขึ้นบรรทัดใหม่ จะมีการแทรก “
” ลงไปในข้อความนั้นๆด้วย เพื่อให้รูปแบบตอนแสดงผลของตัวอักษรมีรูปแบบที่ไม่เป็นปัญหา



รูปที่ 4-10 แสดงโฟลว์ชาร์ตของ `addnew.jsp`

ในการตรวจสอบอินพุตนั้นจะแก้ว่าสมาชิกได้มีการพิมพ์ชื่อกระทู้และรายละเอียดลงมาหรือไม่

4.2.13 หน้าแสดงและตอบกระทู้ (view.jsp)

เมื่อสมาชิกทำการคลิกเมาส์ที่ชื่อกระทู้ภายในหน้า `webboard.jsp` ก็จะทำการเปิดเข้ามาดูว่าภายในกระทู้มีอะไรอยู่บ้าง และสามารถทำการตอบกระทู้นั้นๆได้

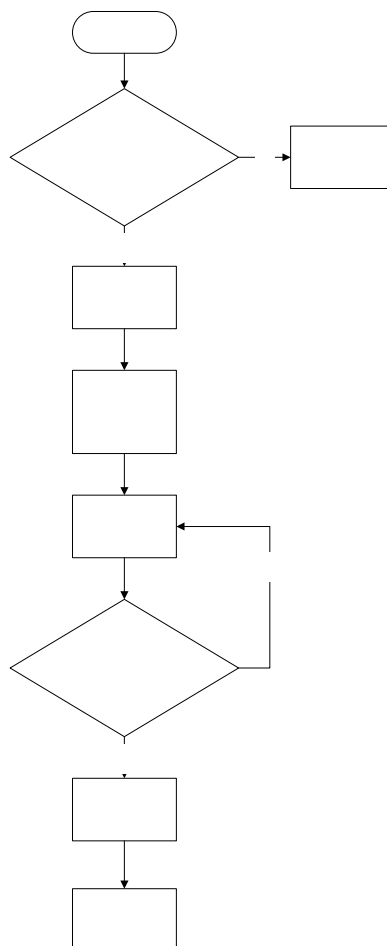
เพจนี้จะทำการเรียกเมธอด `filter_comment` เช่นเดียวกันกับ `addnew.jsp` และเรียกเมธอด `newline` เพิ่มขึ้นมา ซึ่งมีการทำงานดังนี้


```

public static String newline(String input)
{
    StringBuffer temp = new StringBuffer();
    StringTokenizer st = new StringTokenizer(input, "\n");
    while(st.hasMoreTokens())
        temp.append(st.nextToken() + "<br>");
    return(temp.toString());
}

```

คือเนื่องจากการรับข้อความเข้ามาทาง textfield ของเพจ HTML เมื่อเกิดการเว้นบรรทัดขึ้นจะแทนด้วยอักขระ "\n" ซึ่งการที่เราจะนำข้อความนี้ไปแสดงบนเพจ HTML จะต้องทำการแปลงให้เป็น
 ก่อนซึ่งเป็นคำสั่งเว้นบรรทัดจริงๆ ดังนั้นจึงต้องเรียกเมธอดนี้ก่อนการแสดงผล



รูปที่ 4-11 แสดงโฟลว์ชาร์ตของ view.jsp

4.2.14 หน้าแสดงความผิดพลาดของเซิร์ฟเวอร์ (error.jsp)

หน้านี้จะถูกแสดงขึ้นมาเมื่อเกิดความผิดพลาดของเว็บเซิร์ฟเวอร์ขึ้น โดยเว็บเซิร์ฟเวอร์จะเรียกขึ้นมาเองเลย เพื่อเป็นการแจ้งข้อผิดพลาด และปกปิดค่าตัวแปร (parameter) ต่างๆที่ไม่ควรจะให้ผู้ใช้งานเห็นด้วยคำสั่ง

```
<%@ errorPage="error.jsp">
```

4.2.15 ไฟล์เก็บส่วนหัวและแถบเมนูด้านข้าง (header.jsp, member_header.jsp และ

member_left.jsp)

เพื่อการแก้สิ่งที่อาจเกิดขึ้นภายหลังด้วยความง่ายและสะดวก จึงได้ทำการรวมสิ่งที่ต้องมีในแต่ละเพจที่เหมือนกันไว้ในไฟล์เดียว ด้วยคำสั่ง

```
<jsp:include page="header.jsp"/>
```

```
<jsp:include page="member_header.jsp"/>
```

```
<jsp:include page="member_left.jsp"/>
```

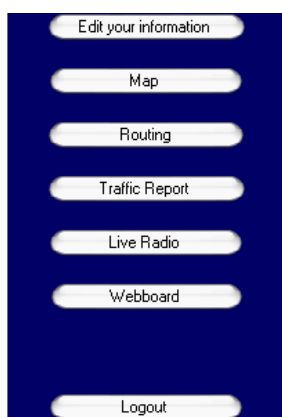
โดยที่ไฟล์ header.jsp จะถูกเรียกใช้โดยเพจที่ผู้ใช้ไม่ได้ผ่านการล็อกอินเข้าสู่ระบบ ส่วน member_header.jsp และ member_left.jsp จะถูกเรียกใช้ในเพจที่ผู้ใช้เป็นสมาชิกและผ่านการล็อกอินแล้ว



รูปที่ 4-12 หน้าตาของ member_header.jsp



รูปที่ 4-13 หน้าตาของ header.jsp



รูปที่ 4-14 หน้าตาของ member_left.jsp

4.2.16 โปรแกรม Java Applet ที่ใช้แสดงแผนที่ (Map.class)

ใช้ในการแสดงและคอยกำกับการแสดงของส่วนที่เป็นรูปแผนที่โดยเฉพาะ คือ จะทำการเลื่อนรูปภาพไปได้ 8 ทิศทาง เลือกรูปภาพที่จะนำมาแสดง ทำการวาดรูปหรือเพิ่มจุดที่เป็นโหนดลงในแผนที่ และจากนั้นก็ทำการแสดงรายละเอียดของบริเวณที่ผู้ต้องการ

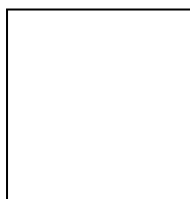
Applet นี้ถูกแบ่งแยกการทำงานออกเป็นหลายๆส่วนซึ่งทำงานไม่เหมือนกัน แต่เกี่ยวข้องกัน โดยแบ่งเป็นเมธอดที่ ซึ่งมีเมธอดที่สำคัญดังนี้คือ

- CountXY ใช้ในการนับจำนวนของโหนดที่มีอยู่ในภาพที่กำลังแสดงอยู่ เพื่อให้โปรแกรมสามารถทำงานได้ถูกต้อง โดยจะเป็นตัวกำหนดจำนวนการวนลูป (For loop) เพื่อการวาดโหนดในรูปนั้นๆ
- getX ใช้ในการดึงค่าคู่ลำดับแกน X ของโหนดแต่ละโหนดในภาพจากฐานข้อมูลที่กำลังแสดงอยู่ เพื่อใช้ในการวาดจุดโหนด
- getY ใช้ในการดึงค่าคู่ลำดับแกน Y ของโหนดแต่ละโหนดในภาพจากฐานข้อมูลที่กำลังแสดงอยู่ เพื่อใช้ในการวาดจุดโหนด
- shownode ใช้ในการแสดงรายละเอียดของโหนดนั้นๆ เมื่อนำเมาส์ไปวางไว้ยังตำแหน่งที่เป็นโหนด
- getPic ใช้ในการติดต่อส่วนที่เป็น JSP คือ เมื่อผู้ใช้ทำการเลือกเขตที่จะแสดงรูปแผนที่ เมธอดนี้จะทำการหาจากฐานข้อมูลว่า จะต้องแสดงรูปใด
- กลุ่มเมธอดที่ใช้แสดงสถานะเมื่อทำการเลื่อนเมาส์
- และกลุ่มเมธอดที่ใช้ในการเปลี่ยนภาพที่แสดงไปในทิศทางต่างๆ

และสำหรับชื่อไฟล์ที่เป็นรูปภาพแผนที่ที่จะต้องทำการเก็บนำลักษณะคู่อันดับ XY เช่น “e10044600n1346000s4.jpg” โดยที่ตัวเลขชุดแรกที่อยู่ทางด้านซ้ายมือนั้นคือค่าของแกน X และถัดมาทางขวามือคือค่าแกน Y โดยที่เมื่อทำการเปลี่ยนภาพที่จะแสดงนั้นค่าเหล่านี้จะบวกหรือลบด้วย 2400 (เมื่อต้องการภาพที่อยู่ด้านบนจะบวกแกน Y ด้วย 2400 เช่นกันทางด้านล่างจะลบด้วย 2400 และสำหรับเมื่อต้องการจะเลื่อนไปทางด้านซ้ายจะลบค่าแกน X ด้วย 2400 เช่นกันทางด้านขวาจะบวกด้วย 2400)

แต่คู่อันดับของโหนดแต่ละโหนดนั้นจะต่างกัน คือ แต่ละภาพจะไม่ขึ้นต่อกัน เช่น ภาพ ก. จะมีคู่อันดับ (x,y) ตั้งแต่ (0,0) ถึง (600,600) และภาพ ข. ก็จะมีตั้งแต่ (0,0) ถึง (600,600) เหมือนกัน

(0,0)



(600,600)

รูปที่ 4-15 รูปแบบคู่อันดับ XY ของโหนด

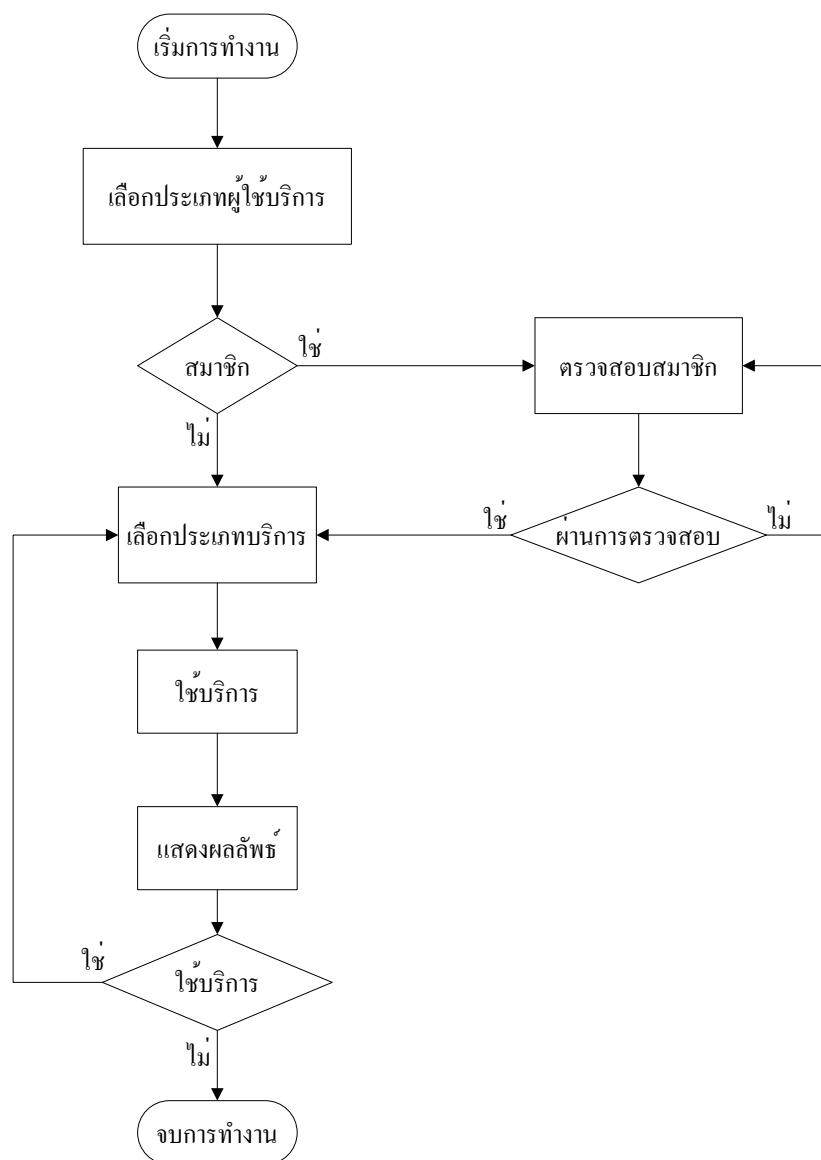
4.3 การทำงานของแอปพลิเคชันค้นหาเส้นทางบนโทรศัพท์เคลื่อนที่

การทำงานของแอปพลิเคชันจะกล่าวถึง ในลักษณะของกิจกรรมที่ผู้ใช้งานกระทำกับตัวระบบ โดยมีเซิร์ฟเวอร์และฐานข้อมูลเป็นส่วนประกอบในการทำงานด้วย ซึ่งสามารถแยกได้ 4 ประเภท ได้แก่ การทำงานของแอปพลิเคชันโดยรวม, การทำงานของบริการค้นหาเส้นทางจราจร, การทำงานของบริการรายงานสภาพการจราจร และการทำงานของบริการแก้ไขข้อมูลสมาชิก

4.3.1 การทำงานของแอปพลิเคชันโดยรวม

ขั้นตอนการทำงานของแอปพลิเคชัน โดยรวมจะเป็นไปตามโฟลว์ชาร์ตของการทำงาน ดังรูปที่ 4-16 ซึ่งมีขั้นตอนดังนี้

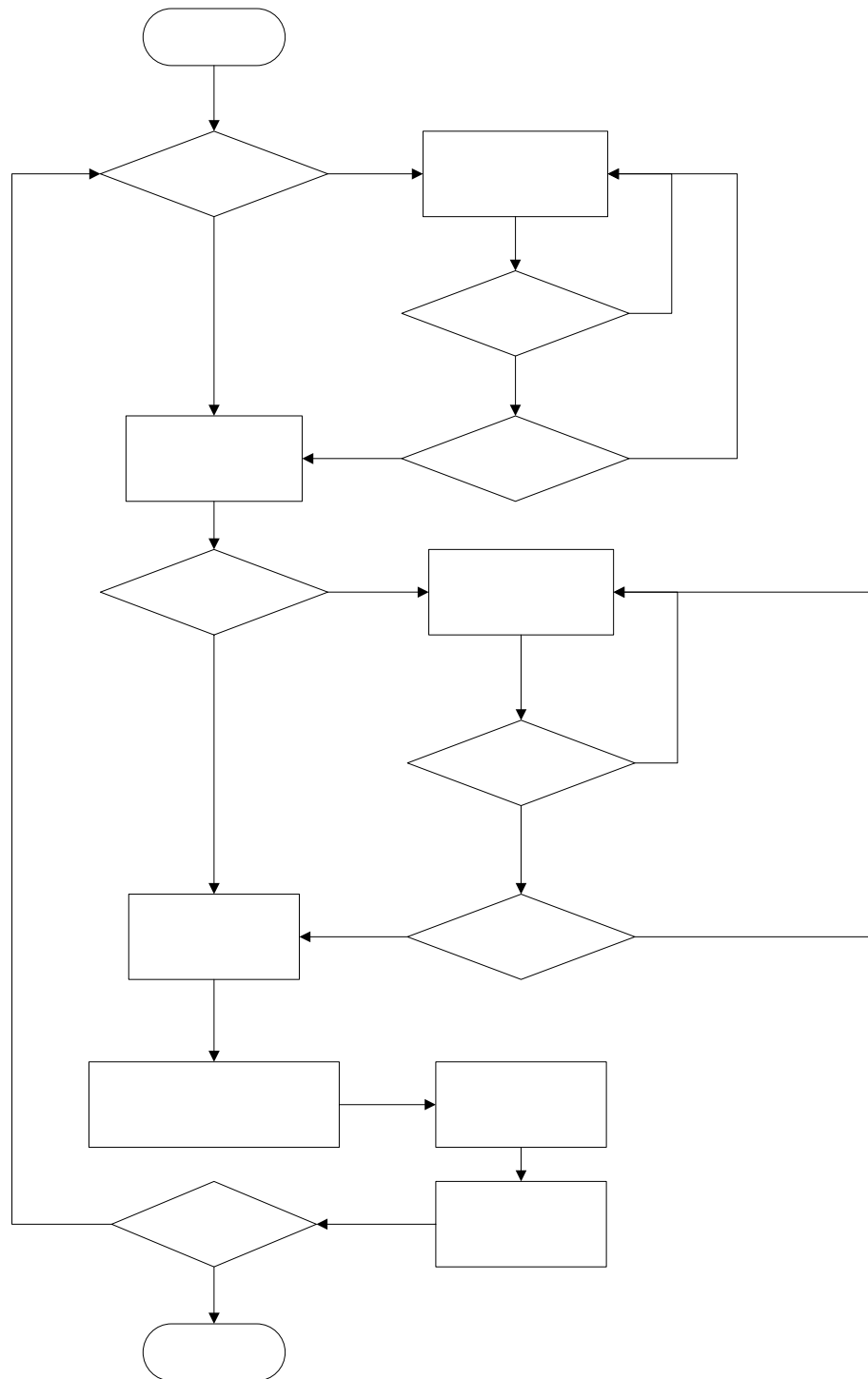
- เมื่อผู้ใช้บริการรันแอปพลิเคชัน ขั้นตอนแรกจะต้องเลือก ประเภทของผู้เข้าใช้งาน ได้แก่ สมาชิก หรือ ผู้ใช้งานทั่วไป เพราะสมาชิกจะแตกต่างกับผู้ใช้งานทั่วไปตรงที่ มีข้อมูลต้นทาง, ปลายทาง และข้อมูลเขตรายงานที่เป็นประโยชน์แก่สมาชิกในบริการค้นหาเส้นทาง, บริการรายงานสภาพการจราจร และบริการแก้ไขข้อมูลสมาชิก
- ถ้าผู้ใช้บริการเป็นสมาชิก จะมีการพิสูจน์การมีตัวตนของสมาชิก โดยให้สมาชิกระบุชื่อสมาชิก และรหัสผ่าน ในขั้นตอนนี้จะมีการติดต่อและส่งข้อมูลร้องขอไปยังเซิร์ฟเวอร์ เพื่อตรวจสอบ และเซิร์ฟเวอร์จะส่งผลลัพธ์กลับมายังแอปพลิเคชัน
- ถ้าผลลัพธ์ที่ได้ยืนยันว่าเป็นสมาชิกจริง ผู้ใช้จะเข้าสู่การให้บริการในฐานะสมาชิก ถ้าผลลัพธ์ที่ได้ระบุว่าไม่มีชื่อสมาชิกหรือรหัสผ่านที่ตรงกับในฐานข้อมูล โปรแกรมจะให้ผู้ใช้ระบุชื่อสมาชิก และรหัสผ่านอีกครั้ง
- ก่อนที่สมาชิกจะเข้าสู่การให้บริการ โปรแกรมจะทำการติดต่อกับเซิร์ฟเวอร์อีกครั้งเพื่อร้องขอข้อมูลทั้งหมดของสมาชิก และนำมาเก็บไว้ในอาร์เรย์สตริง
- ทั้งผู้ใช้งานทั่วไปและสมาชิกที่ผ่านการตรวจสอบแล้วจะเข้ามาสู่การรับบริการ ที่มีประเภทของบริการให้เลือก โดยสำหรับผู้ใช้งานทั่วไปจะมีบริการค้นหาเส้นทางและบริการรายงานสภาพการจราจร และสำหรับสมาชิกจะเพิ่มบริการแก้ไขข้อมูลสมาชิกอีก 1 บริการ
- เมื่อผู้ใช้งานเลือกบริการแล้ว จะใช้บริการต่างๆ เพื่อให้ได้ผลลัพธ์ของบริการ โดยสำหรับสมาชิกจะมีข้อมูลต้นทาง ปลายทางและ เขตรายงาน ที่เป็นประโยชน์กับการใช้บริการ
- หลังจากที่ได้รับผลลัพธ์ของบริการแล้ว ผู้ใช้สามารถที่จะเข้าใช้บริการได้อีก หรือจะจบการทำงานแอปพลิเคชันก็ได้



รูปที่4-16 แสดงโฟลว์ชาร์ต การทำงานของระบบโดยรวม

4.3.2 การทำงานของบริการค้นหาเส้นทางจราจร

ขั้นตอนการทำงานของบริการค้นหาเส้นทาง จะรวมการทำงานของผู้ใช้ที่เป็นสมาชิกกับผู้ใช้ทั่วไปเข้าด้วยกัน ซึ่งสมาชิกจะมีเส้นทางกับปลายทางให้อยู่แล้ว โดยการทำงานจะเป็นไปตามโฟลว์ชาร์ตดังรูปที่ 4-17 ซึ่งมีขั้นตอนดังนี้



รูปที่4-17 แสดงโฟลว์ชาร์ตการทำงานของบริการค้นหาเส้นทางจราจร

- จุดเริ่มต้นของการใช้บริการ คือ การเลือกเส้นทาง โดยถ้าเป็นผู้ใช้งานทั่วไปต้องค้นหารายการเส้นทางโดยอาศัยเว็บไซต์ของการค้นหา สำหรับสมาชิก ชื่อเส้นทางจะมีให้อยู่แล้วสามารถเลือกได้

พื้นที่ซึ่งโปรแกรมจะดึงข้อมูลต้นทางจากอาเรย์มาแสดงผลเป็นลำดับรายการ หรือจะค้นหารายการต้นทางใหม่ก็ได้

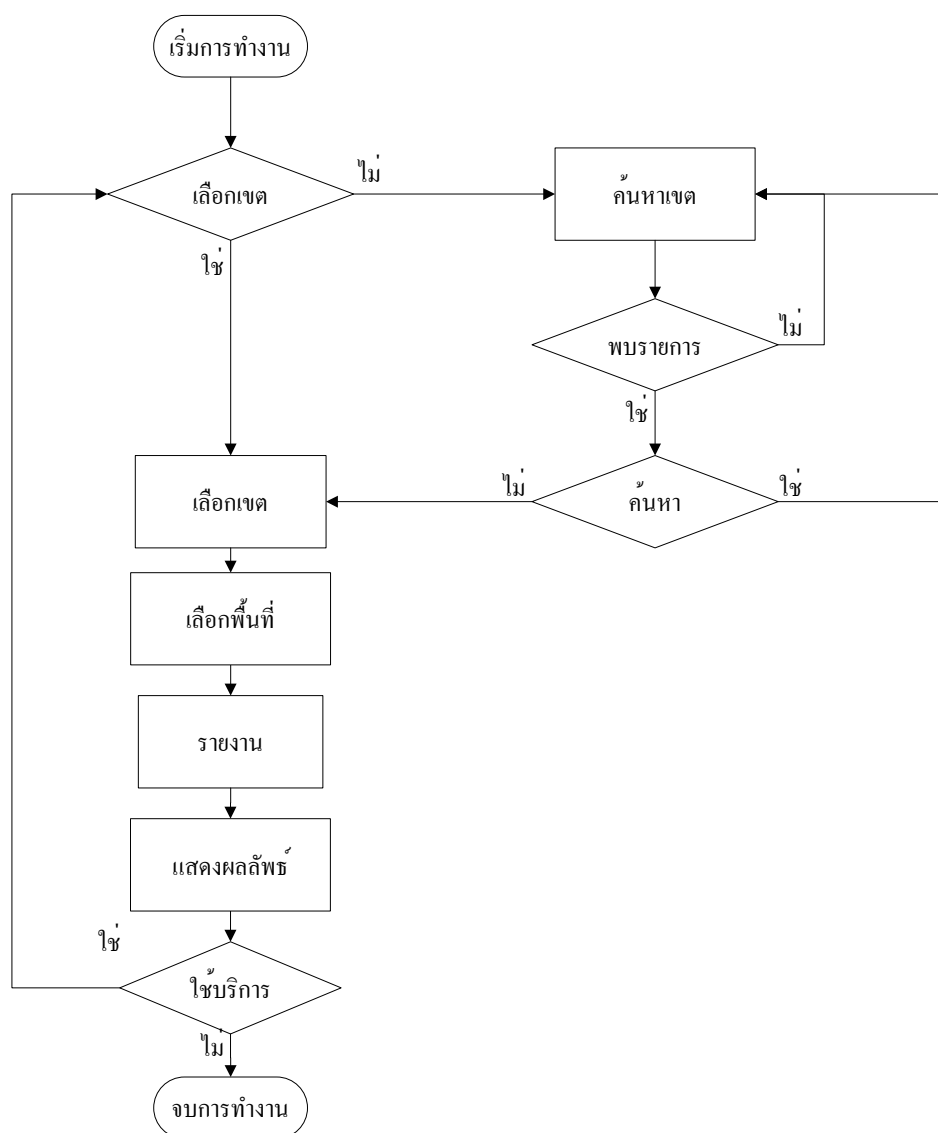
- การค้นหาคือการติดต่อและส่งข้อมูลไปยังเซิร์ฟเวอร์ ถ้าการค้นหาสำเร็จเซิร์ฟเวอร์จะส่งข้อมูลที่เป็นลำดับรายการต้นทางให้ผู้ใช้เลือก
- ถ้าหากไม่พบรายการ โปรแกรมจะให้ผู้ใช้ระบุคีย์เวิร์ดและค้นหารายการต้นทางใหม่
- ผู้ใช้งานสามารถเลือกที่จะค้นหารายการได้มากกว่า 1 ครั้ง
- เมื่อผู้ใช้เลือกต้นทางได้แล้ว โปรแกรมจะนำมาสู่ขั้นตอนการเลือกปลายทาง โดยมีขั้นตอนการติดต่อเหมือนกับการเลือก และค้นหาปลายทางเช่นกัน และสำหรับสมาชิก ชื่อปลายทางจะมีให้อยู่แล้วสามารถเลือกได้ทันที
- ขั้นตอนต่อมาคือการเลือกปัจจัยในการค้นหาเส้นทาง ซึ่งมีอยู่ 3 ประเภท คือ ผู้ใช้ต้องการเส้นทางที่มีค่าใช้จ่าย เช่น เสียค่าทางด่วน, ค่าน้ำมัน น้อยที่สุด เป็นต้น ผู้ใช้ต้องการเส้นทางที่ใช้เวลาในการเดินทางน้อยที่สุด หรือ ผู้ใช้จะต้องการเส้นทางที่มีระยะทางสั้นที่สุด โดยสามารถเลือกได้มากกว่า 1 ปัจจัย
- หลังจากนั้นแอปพลิเคชันจะทำการส่งข้อมูล ต้นทางและปลายทางไปคำนวณที่ฝั่งเซิร์ฟเวอร์
- เซิร์ฟเวอร์คำนวณหาเส้นทางที่ดีที่สุดแล้วจึงส่งผลลัพธ์กลับมาให้ที่ฝั่งไคลเอนต์
- หลังจากที่ได้รับผลลัพธ์แล้ว ผู้ใช้สามารถที่จะเข้าใช้บริการได้อีก หรือจะจบการทำงาน แอปพลิเคชันก็ได้

4.3.3 การทำงานของบริการรายงานสภาพการจราจร

ขั้นตอนการทำงานของบริการรายงานสภาพจราจร จะรวมการทำงานของผู้ใช้ที่เป็นสมาชิกกับผู้ใช้ทั่วไปเข้าด้วยกัน ซึ่งสมาชิกจะมีเขตรายงานให้อยู่แล้ว โดยการทำงานจะเป็นไปตามโฟลว์ชาร์ตดังรูปที่ 4-18 ซึ่งมีขั้นตอนดังนี้

- จุดเริ่มต้นของการใช้บริการ คือ การเลือกเขตรายการ โดยถ้าเป็นผู้ใช้งานทั่วไปต้องค้นหารายการเขตโดยอาศัยคีย์เวิร์ดของการค้นหา สำหรับสมาชิก ชื่อเขตจะมีให้อยู่แล้วสามารถเลือกได้ทันที ซึ่งโปรแกรมจะดึงข้อมูลเขตรายงานจากอาเรย์มาแสดงผลเป็นลำดับรายการ หรือจะค้นหารายการเขตใหม่ก็ได้
- การค้นหาคือการติดต่อและส่งข้อมูลไปยังเซิร์ฟเวอร์ ถ้าการค้นหาสำเร็จเซิร์ฟเวอร์จะส่งข้อมูลที่เป็นลำดับรายการเขตให้ผู้ใช้เลือก
- ถ้าหากไม่พบรายการ โปรแกรมจะให้ผู้ใช้ระบุคีย์เวิร์ดและค้นหารายการเขตใหม่
- ผู้ใช้งานสามารถเลือกที่จะค้นหารายการได้มากกว่า 1 ครั้ง
- เมื่อผู้ใช้เลือกเขตรายงานได้แล้ว โปรแกรมจะมีการติดต่อกับเซิร์ฟเวอร์อีกครั้งเพื่อขอข้อมูลพื้นที่ต่างๆ ในเขตรายงาน ซึ่งจะนำมาสู่ขั้นตอนการเลือกพื้นที่

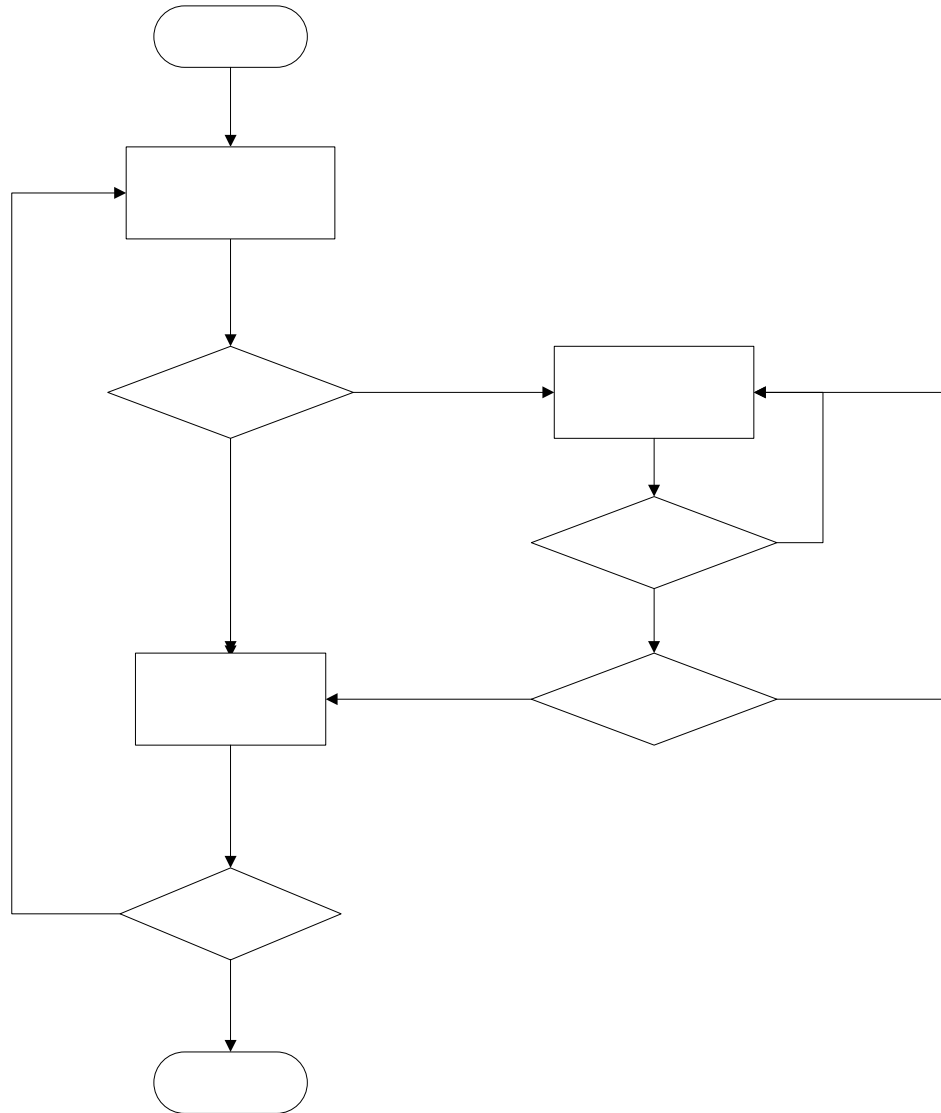
- ขั้นตอนต่อมาคือการร้องขอข้อมูลรายงานสภาพจราจรในพื้นที่ที่ได้เลือกไว้ แอปพลิเคชันจะทำการส่งข้อมูลพื้นที่รายงานไปที่ฝั่งเซิร์ฟเวอร์เพื่อให้ได้ผลลัพธ์ออกมา
- เซิร์ฟเวอร์ค้นหาหารายงานสภาพจราจรจากฐานข้อมูลแล้วจึงส่งผลลัพธ์การรายงานกลับมาให้ที่ฝั่งไคลเอนต์
- หลังจากที่ได้รับผลลัพธ์แล้ว ผู้ใช้สามารถที่จะเข้าใช้บริการได้อีก หรือจะจบการทำงาน แอปพลิเคชันก็ได้



รูปที่4-18 แสดงโฟลว์ชาร์ตการทำงานของบริการรายงานสภาพจราจร

4.3.4 การทำงานของบริการแก้ไขข้อมูลสมาชิก

ขั้นตอนการทำงานของบริการแก้ไขข้อมูล จะเป็นบริการที่เพิ่มเข้ามาสำหรับสมาชิกเท่านั้น โดยการทำงานจะเป็นไปตามฟลว์ชาร์ตดังรูปที่ 4-19 ซึ่งมีขั้นตอนดังนี้



รูปที่4-19 แสดงฟลว์ชาร์ตการทำงานของบริการแก้ไขข้อมูลสมาชิก

- จุดเริ่มต้นของการใช้บริการ คือ การแก้ไขข้อมูลทั่วไปในแต่ละฟิลด์ ซึ่งประกอบด้วย ข้อมูลชื่อสมาชิก, รหัสผ่าน, ชื่อสมาชิกจริง, อีเมล, ที่อยู่, ชื่อผู้ให้บริการ, เบอร์โทรศัพท์เคลื่อนที่, ชื่อผู้ใช้งานโทรศัพท์, รหัสผ่านโทรศัพท์

- จากนั้นเป็นการแก้ไขข้อมูลเขตของสมาชิกซึ่งอาจจะต้องมีการค้นหาเพื่อหารายการเขตโดยอาศัยคีย์เวิร์ดของการค้นหา สำหรับสมาชิก ชื่อเขตจะมีให้อยู่แล้วสามารถเลือกได้ทันทีซึ่งโปรแกรมจะดึงข้อมูลเขตรายงานจากอาเรย์มาแสดงผลเป็นลำดับรายการ หรือจะค้นหารายการเขตใหม่ก็ได้
- การค้นหาคือการติดต่อและส่งข้อมูลคีย์เวิร์ดไปยังเซิร์ฟเวอร์ ถ้าการค้นหาสำเร็จเซิร์ฟเวอร์จะส่งข้อมูลที่เป็นลำดับรายการเขตให้ผู้ใช้เลือก
- ถ้าหากไม่พบรายการ โปรแกรมจะให้ผู้ใช้นัดคีย์เวิร์ดและค้นหาใหม่
- ผู้ใช้งานสามารถเลือกที่จะค้นหาได้รายการมากกว่า 1 ครั้ง
- เมื่อผู้ใช้แก้ไขข้อมูลทุกอย่างครบแล้ว โปรแกรมจะมีการติดต่อกับเซิร์ฟเวอร์เพื่อส่งข้อมูลทั้งหมด ไปอัพเดทฐานข้อมูล

หลังจากที่แก้ไขข้อมูลเสร็จแล้ว ผู้ใช้สามารถที่จะเข้าใช้บริการได้อีก หรือจะจบการทำงานแอปพลิเคชันก็ได้

4.4 การออกแบบแอปพลิเคชันบนโทรศัพท์เคลื่อนที่

แอปพลิเคชันจะถูกเขียนขึ้นและใช้ในฝั่งไคลเอนต์โดยภาษาที่ใช้หลักการของ Object Orientation ซึ่งมีการเขียนคลาสออกเป็นหลายคลาส เมธอดและตัวแปรต่างๆ จะถูกเรียกใช้ผ่านออบเจกต์ที่ได้ประกาศไว้ การออกแบบคลาสจะออกแบบตามประเภทของหน้าการแสดงผล เช่น คลาส UserScreen สำหรับแสดงหน้าการเลือกประเภทของผู้ใช้บริการ เป็นต้น ทั้งนี้เพื่อแยกหน้าที่การทำงานในแต่ละหน้าอย่างชัดเจน ซึ่งประกอบด้วยคลาสทั้งหมด ดังนี้

4.4.1 คลาส TrafficReport

เป็นคลาสหลักที่มีการประกาศเมธอดต่างๆ ที่ใช้สำหรับประมวลผล และการแสดงผลออกทางจอภาพของโทรศัพท์เคลื่อนที่ โดยการแสดงผลจะใช้การประกาศออบเจกต์ของคลาสอื่นๆ ร่วมกับคลาส Display ด้วย เมธอดต่างๆ ในคลาส TrafficReport มีดังนี้

- เมธอด TrafficReport() เป็นคอนสตรัคเตอร์ของคลาสที่ไม่มีการใช้งาน
- เมธอด StartApp() เป็นเมธอดของ MIDlet ใช้เพื่อจองทรัพยากรของระบบ และถือเป็นจุดเริ่มของโปรแกรมด้วย เมื่อ MIDlet เริ่มทำงานเมธอดนี้จะถูกเรียกขึ้นมา
- เมธอด PauseApp() เป็นเมธอดของ MIDlet ที่บอก MIDlet ให้พักการทำงานชั่วคราวและคืนทรัพยากรสู่ระบบ

- **เมธอด DestroyApp()** เป็นเมธอดของ MIDlet เพื่อจบการทำงานของ MIDlet และคืนทรัพยากรทั้งหมดกลับสู่ระบบ
- **เมธอด goUser()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส UserScreen เพื่อแสดงหน้าการเลือกประเภทผู้ให้บริการ ออกทางจอภาพ
- **เมธอด goLogin()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส LoginScreen เพื่อแสดงหน้าการตรวจสอบความเป็นสมาชิกของผู้ใช้งานระบบ ออกทางจอภาพ
- **เมธอด goMember()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส MemberScreen เพื่อแสดงหน้าการเลือกประเภทของบริการสำหรับสมาชิก ออกทางจอภาพ โดยในคลาสนี้จะมีการติดต่อกับเซิร์ฟเวอร์เพื่อร้องขอข้อมูลของสมาชิก
- **เมธอด goNonmember()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส NonmemberScreen เพื่อแสดงหน้าการเลือกประเภทของบริการสำหรับผู้ทั่วไปที่ไม่ใช่สมาชิก ออกทางจอภาพ
- **เมธอด goSearchsource()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส SearchsourceScreen เพื่อแสดงหน้าการเลือกและค้นหาเส้นทาง ออกทางจอภาพ
- **เมธอด goSearchdest()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส SearchdestScreen เพื่อแสดงหน้าการเลือกและค้นหาปลายทาง ออกทางจอภาพ
- **เมธอด goCost()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส CostScreen เพื่อแสดงหน้าการเลือกปัจจัยในการค้นหาเส้นทาง ออกทางจอภาพ
- **เมธอด goReport()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส ReportareaScreen เพื่อแสดงหน้าการเลือกและค้นหาเขตรายงาน ออกทางจอภาพ
- **เมธอด goRange()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส ReportrangeScreen เพื่อแสดงหน้าการเลือกพื้นที่ของเขตรายงาน ออกทางจอภาพ
- **เมธอด goResponse(String input)** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส ResponseScreen เพื่อแสดงผลลัพธ์เนื่องจากการค้นหาเส้นทางและการรายงานสภาพจราจรในพื้นที่เขตออกทางจอภาพ
- **เมธอด goEdituser()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส EdituserScreen เพื่อแสดงหน้าการแก้ไขข้อมูลของสมาชิก ออกทางจอภาพ
- **เมธอด goEditdistrict()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส EditdistrictScreen เพื่อแสดงหน้าการแก้ไขข้อมูลเขตรายงาน ออกทางจอภาพ
- **เมธอด goEditok()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส Alert เพื่อแสดงหน้าของการแก้ไขข้อมูลสมาชิกสำเร็จโดยจะแสดงออกมาชั่วระยะเวลาหนึ่งก่อนที่จะเข้าสู่หน้าประเภทของบริการ

- **เมธอด goError(String flag)** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส Alert เพื่อแสดงหน้าความผิดพลาดอันเนื่องมาจากบริการต่างๆ ออกทางจอภาพ โดยในหน้าของความผิดพลาดจะแสดงในช่วงระยะเวลาหนึ่งก่อนที่จะเข้าสู่หน้าประเภทของบริการ
- **เมธอด goConnect(String input)** เป็นเมธอดที่ใช้สำหรับส่งข้อมูลร้องขอและรับข้อมูลตอบรับต่างๆ ระหว่างผู้ใช้งานโทรศัพท์เคลื่อนที่และเครื่องเซิร์ฟเวอร์ผู้ให้บริการ

4.4.2 คลาส UserScreen

เป็นคลาสเพื่อแสดงประเภทของผู้ใช้บริการ คือ ผู้ใช้บริการที่เป็นสมาชิก และผู้ให้บริการทั่วไป สืบทอดมาจากคลาส List ที่แสดงเป็นลำดับรายการให้ผู้ใช้งานเลือกได้ ประกอบด้วยตัวแปรและเมธอดต่างๆ ที่สำคัญ ดังนี้

- **ตัวแปร userStatus** เป็นตัวแปรชนิด บูลีน ใช้สำหรับเก็บสถานะของผู้ใช้ ปกติเมื่อโปรแกรมทำงานค่า userstatus จะถูกกำหนดให้เป็น false ซึ่งจะหมายถึงผู้ใช้งานในขณะนี้เป็นผู้ใช้งานทั่วไป และ userstatus จะถูกกำหนดเป็น true สำหรับสมาชิก
- **เมธอด UserScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ลำดับรายการเลือก และ ปุ่ม Select
- **เมธอด commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส UserScreen

4.4.3 คลาส LoginScreen

เป็นคลาสที่แสดงหน้าของการเข้าสู่ระบบของสมาชิก สืบทอดคุณสมบัติมาจากคลาส Form โดยคลาส LoginScreen จะมีตัวแปรและเมธอดที่สำคัญ ดังนี้

- **ตัวแปร usr_pass** เป็นตัวแปรชนิดสตริง ใช้สำหรับเก็บข้อมูลชื่อและรหัสผ่านของผู้ใช้ เพื่อใช้ในการร้องขอข้อมูลสมาชิกที่เครื่องเซิร์ฟเวอร์
- **เมธอด LoginScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิวด์ข้อมูลชื่อผู้ใช้, ฟิวด์ข้อมูลรหัสผ่าน ปุ่ม Select และปุ่ม Back
- **เมธอด commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส LoginScreen

4.4.4 คลาส MemberScreen

เป็นคลาสแสดงหน้าบริการสำหรับสมาชิก ที่สืบทอดมาจากคลาส List ได้แก่ บริการค้นหาเส้นทาง, บริการรายงานสภาพจราจร และบริการการแก้ไขข้อมูลสมาชิก ซึ่งเป็นบริการเฉพาะสำหรับสมาชิกเท่านั้น คลาส MemberScreen จะมีตัวแปรและเมธอดที่สำคัญดังนี้

- **ตัวแปร infoList** เป็นตัวแปรชนิดอาร์เรย์ แบบสตริง ขนาด 14 ช่องที่ใช้สำหรับเก็บข้อมูลต่างๆ ของสมาชิก ได้แก่ ชื่อสมาชิก, รหัสผ่าน, ชื่อจริง, อีเมลล์, ที่อยู่, ชื่อเขตรายงาน, ผู้ให้บริการ โทรศัพท์, เบอร์โทรศัพท์, ชื่อผู้ใช้โทรศัพท์, รหัสผ่านโทรศัพท์, ดันทาง1, ดันทาง2, ปลายทาง1 และ ปลายทาง2 สำหรับข้อมูลเขตรายงาน, ดันทางและปลายทาง จะช่วยเพิ่มความสะดวกให้กับสมาชิกในกรณีที่สมาชิกต้องการค้นหาเส้นทางจราจรที่ไปเป็นประจำ หรือต้องการทราบรายงานสภาพจราจรในเขตพื้นที่รายงานเดิม สมาชิกไม่ต้องเสียเวลากับการค้นหารายชื่อ
- **เมธอด MemberScreen(TrafficReport midlet, String input)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ List ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, รายการบริการต่างๆ และ ปุ่ม Select รวมถึงการนำผลลัพธ์ของข้อมูลสมาชิกที่ได้จากเครื่องเซิร์ฟเวอร์มาเก็บลงในอาร์เรย์
- **เมธอด commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส MemberScreen

4.4.5 คลาส NonmemberScreen

เป็นคลาสแสดงหน้าบริการสำหรับผู้ใช้งานทั่วไป สืบทอดมาจากคลาส List ได้แก่ บริการค้นหาเส้นทาง, บริการรายงานสภาพจราจร NonmemberScreen จะมีเมธอดที่สำคัญดังนี้

- **เมธอด NonmemberScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ List ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, รายการบริการสำหรับผู้ใช้งานทั่วไป
- **เมธอด commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส NonmemberScreen

4.4.6 คลาส SearchsourceScreen

เป็นคลาสแสดงหน้าบริการค้นหาดันทางจราจร สืบทอดมาจากคลาส Form เพื่อวางคอมโพเนนต์ชนิด Item ได้หลากหลาย SearchsourceScreen จะมีตัวแปรและเมธอดที่สำคัญดังนี้

- **ตัวแปร sourceList** เป็นตัวแปรชนิดอาร์เรย์ แบบสตริง ขนาด 50 ช่องใช้สำหรับเก็บข้อมูลรายชื่อดันทางที่ค้นหาเพื่อเตรียมมาวางเป็นลำดับรายการ

- ตัวแปร `routeArray` เป็นตัวแปรชนิดอาร์เรย์ แบบสตริง ขนาด 5 ช่อง ใช้สำหรับเก็บข้อมูลต้นทาง, ปลายทางและปัจจัยในการค้นหาเส้นทาง เพื่อใช้ในการค้นหาเส้นทางจากต้นทางไปยังปลายทางโดยคำนึงถึงปัจจัยที่เลือก โดยจะส่งเป็นข้อมูลร้องขอไปยังเครื่องเซิร์ฟเวอร์
- เมธอด `SearchsourceScreen(TrafficReport midlet)` เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลสำหรับใช้ค้นหารายการชื่อต้นทาง, คอมโพเนนต์ ประเภทแสดงลำดับรายการ, ปุ่ม Select, ปุ่ม Search และปุ่ม Back
- เมธอด `commandAction(Command c, Displayable d)` เป็นเมธอดที่มีอยู่ในแฟกเกจของ `j2me` อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส `SearchsourceScreen`

4.4.7 คลาส `SearchdestScreen`

เป็นคลาสแสดงหน้าบริการค้นหาปลายทางจราจร สืบทอดมาจากคลาส `Form` เพื่อวางคอมโพเนนต์ชนิด `Item` ให้หลากหลาย `SearchdestScreen` จะมีตัวแปรและเมธอดที่สำคัญดังนี้

- ตัวแปร `destList` เป็นตัวแปรชนิดอาร์เรย์ แบบสตริง ขนาด 50 ช่องใช้สำหรับเก็บข้อมูลรายชื่อปลายทางที่ค้นหาเพื่อเตรียมมาวางเป็นลำดับรายการ
- เมธอด `SearchdestScreen(TrafficReport midlet)` เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลสำหรับใช้ค้นหาชื่อปลายทาง, คอมโพเนนต์ประเภทลำดับรายการ สำหรับแสดงรายการชื่อปลายทาง, ปุ่ม Select, ปุ่ม Search และปุ่ม Back
- เมธอด `commandAction(Command c, Displayable d)` เป็นเมธอดที่มีอยู่ในแฟกเกจของ `j2me` อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส `SearchdestScreen`

4.4.8 คลาส `CostScreen`

เป็นคลาสแสดงหน้ารายการเลือกปัจจัยในการค้นหาเส้นทาง ได้แก่ ค่าใช้จ่าย, เวลา และระยะทางตามความต้องการในการค้นหาเส้นทางที่แตกต่างกันของผู้ใช้บริการ สืบทอดมาจากคลาส `List` โดยมีเมธอดที่สำคัญดังนี้

- เมธอด `CostScreen(TrafficReport midlet)` เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, คอมโพเนนต์ประเภท `Choice Group` สำหรับแสดงรายการปัจจัยที่มีผลต่อการค้นหาเส้นทาง, ปุ่ม Route และปุ่ม Back
- เมธอด `commandAction(Command c, Displayable d)` เป็นเมธอดที่มีอยู่ในแฟกเกจของ `j2me` อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส `CostScreen`

4.4.9 คลาส ReportareaScreen

เป็นคลาสที่ใช้สำหรับการแสดงผลหน้ารายการเพื่อให้ผู้ใช้สามารถเลือกเขตที่ต้องการทราบสภาพจราจรในขณะนั้น สืบทอดมาจากคลาส Form มีตัวแปรและเมธอดที่สำคัญ ดังนี้

- ตัวแปร **areaList** เป็นตัวแปรอาร์เรย์แบบสตริงขนาด 50 ช่อง สำหรับเก็บรายการเขตที่ค้นหาได้เพื่อนำไปแสดงผล โดยข้อมูลที่ได้อาจจะได้อาจมาจากการร้องขอที่เครื่องเซิร์ฟเวอร์
- เมธอด **ReportareaScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, คอมโพเนนต์ Choice Group แสดงลำดับรายการเขต, ฟิลด์ข้อมูลรหัสผ่าน ปุ่ม Select, ปุ่ม Search และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส ReportareaScreen

4.4.10 คลาส ReportrangeScreen

เป็นคลาสสำหรับการแสดงผลหน้ารายการพื้นที่ที่มีการรายงานสภาพจราจรในขณะนั้น โดยจะแสดงในลักษณะของลำดับรายการ มีคุณสมบัติสืบทอดมาจากคลาส List มีตัวแปรและเมธอดที่สำคัญดังนี้

- เมธอด **ReportrangeScreen(TrafficReport midlet, String input)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ List ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลผลลัพธ์, ปุ่ม Report และปุ่ม Back รวมถึงขั้นตอนการนำข้อมูลผลลัพธ์มาใส่ในฟิลด์ข้อมูล
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส ReportrangeScreen

4.4.11 คลาส ResponseScreen

เป็นคลาสที่ใช้ในการแสดงหน้าผลลัพธ์เนื่องจากการค้นหาเส้นทางหรือการรายงานสภาพการจราจร โดยจะรับข้อมูลเข้ามาและนำข้อมูลไปแสดงผลออกทางจอภาพ คลาส ResponseScreen จะมีคุณสมบัติสืบทอดมาจากคลาส Form โดยจะมีเมธอดต่างๆที่สำคัญดังนี้

- เมธอด **ResponseScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลผลลัพธ์, ปุ่ม Service และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส ResponseScreen

4.4.12 คลาส EdituserScreen

เป็นคลาสใช้สำหรับสมาชิกในกรณีที่สมาชิกเรียกใช้บริการการแก้ไขข้อมูล โดยคลาสจะเป็นส่วนแสดงผลหน้าฟิลด์ข้อมูลต่างๆ เกี่ยวกับสมาชิกทั้งหมด เช่น ข้อมูลชื่อ, ข้อมูลที่อยู่ และข้อมูลชื่ออีเมล ทั้งนี้เพื่อให้สมาชิกสามารถแก้ไขข้อมูลบนโทรศัพท์เคลื่อนที่ได้ มีตัวแปรและเมธอดต่างๆ ที่สำคัญ ดังนี้

- ตัวแปร **pmet** เป็นตัวแปรอาร์เรย์ ชนิดสตริงขนาด 13 ช่อง ใช้สำหรับเก็บข้อมูลใหม่ที่สมาชิกแก้ไขแล้ว ยกเว้นข้อมูลเขตรายงานที่สมาชิกจะต้องเลือกในหน้าแก้ไขข้อมูลเขตรายงานซึ่งแยกออกไปต่างหาก ก่อนที่จะส่งไปเปลี่ยนแปลงที่ฐานข้อมูลบนเครื่องเซิร์ฟเวอร์
- เมธอด **EdituserScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลค้นหารายการเขต, คอมโพเนนต์ลำดับรายการ ปุ่ม Next และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส EdituserScreen

4.4.13 คลาส EditdistrictScreen

เป็นคลาสใช้สำหรับสมาชิกในกรณีที่สมาชิกเรียกใช้บริการการแก้ไขข้อมูล โดยคลาสจะเป็นส่วนแสดงผลหน้าฟิลด์ข้อมูลเฉพาะข้อมูลเขตรายงานเท่านั้น เนื่องจากจะมีลำดับรายการเขตให้สมาชิกสามารถเลือกได้ ซึ่งประหยัดเวลาในการพิมพ์ชื่อให้ถูกต้อง คลาส EditdistrictScreen สืบทอดมาจากคลาส Form โดยมีเมธอดต่างๆ ที่สำคัญ ดังนี้

- เมธอด **EditdistrictScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลค้นหารายการเขตรายงาน, คอมโพเนนต์ลำดับรายการเขต, ปุ่ม Edit, ปุ่ม Search และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส EditdistrictScreen

4.5 การออกแบบไฟล์ประมวลผลบนฝั่งเซิร์ฟเวอร์ของโทรศัพท์เคลื่อนที่

ที่ฝั่งเซิร์ฟเวอร์จะเขียนเป็นไฟล์ JSP ทั้งหมด 9 ไฟล์ โดยแต่ละไฟล์จะทำหน้าที่ในการติดต่อไปยังฐานข้อมูล ด้วยคำสั่งในการค้นหาข้อมูล ที่แตกต่างกัน จากนั้นจะนำค่าข้อมูลที่ได้ส่งกลับไปฝั่งไคลเอนต์เพื่อใช้ประมวลผลต่อไป ไฟล์ต่างๆ มีดังนี้

4.5.1 ไฟล์ Server_Authen.jsp

เป็นไฟล์สำหรับการตรวจสอบความเป็นสมาชิกของผู้ใช้งาน ทำหน้าที่ในการรับข้อมูล ชื่อสมาชิกและรหัสผ่าน และใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลใน ฟิวด์ชื่อสมาชิก และฟิวด์รหัสผ่านตรงกับข้อมูลที่ได้รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะมีค่ารีเทิร์นกลับมาเป็นเลข 1 เซิร์ฟเวอร์จะส่งค่ากลับไปเป็นสตริงว่า “ok” ถ้าค้นหาไม่พบ ผลลัพธ์ที่ได้จะมีค่ารีเทิร์นกลับมาเป็นเลขอื่นๆ ที่ไม่ใช่เลข 1 และเซิร์ฟเวอร์จะส่งค่ากลับไปเป็นสตริงว่า “no”

```
ResultSet myresult = stmt.executeQuery("SELECT count(*) as num FROM member WHERE
usr='"+requirer+"' and pass='"+reqpass+"'"); //set sql command to search username and password field
while(myresult.next()) {
    if(myresult.getInt("num")==1)
        out.println("ok");           //search complete return ok
    else
        out.println("no");           //search fail return no
}
```

4.5.2 ไฟล์ Server_EditSearch.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลต่างๆ ที่เกี่ยวกับสมาชิก เพื่อใช้ในบริการประเภทต่างๆ เช่น บริการค้นหาเส้นทาง จะมีข้อมูลค้นหาและปลายทางปรากฏให้สมาชิกได้เลือกอยู่แล้ว เป็นการประหยัดเวลาในการเลือกการค้นหาและปลายทาง ไฟล์จะทำหน้าที่ในการรับข้อมูล ชื่อสมาชิกและรหัสผ่าน จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลใน ฟิวด์ชื่อสมาชิกและฟิวด์รหัสผ่านตรงกับข้อมูลที่ได้รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของข้อมูลสมาชิกส่งกลับไปฝั่งไคลเอนต์เพื่อนำไปใช้งานต่อไป ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลสมาชิก

```
ResultSet myresult = stmt.executeQuery("SELECT * FROM member WHERE usr='"+requirer+"' and
pass='"+reqpass+"'"); //set sql command to search member information
if(myresult != null) {
    while(myresult.next()) {
        output = myresult.getString("usr") +','+ //get information in each field and-
myresult.getString("pass")+','+ //concatenate with the original output
myresult.getString("name")+','+
myresult.getString("email")+','+
myresult.getString("district")+','+
myresult.getString("address")+','+

```

```

        myresult.getString("mobile")+','+
        myresult.getString("provider")+','+
        myresult.getString("mo_usr")+','+
        myresult.getString("mo_pass")+','+
        myresult.getString("start1")+','+
        myresult.getString("dest1")+','+
        myresult.getString("start2")+','+
        myresult.getString("dest2")+','+$";
    }

    out.println(output);    //return to client
}

```

4.5.3 ไฟล์ Server_RouteSearch.jsp

เป็นไฟล์สำหรับการค้นหารายการข้อมูล เส้นทางหรือปลายทาง ในกรณีที่ผู้ใช้บริการไม่ทราบชื่อของเส้นทางหรือปลายทางที่ต้องการ ไฟล์จะทำหน้าที่ในการรับข้อมูลที่เป็นคีย์เวิร์ดในการค้นหา จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหารายการเส้นทางหรือปลายทางที่มีตัวอักษรในข้อมูลตรงกับคีย์เวิร์ด ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของข้อมูลรายการเส้นทางหรือปลายทาง ส่งกลับไปที่ฝั่งไคลเอนต์เพื่อนำไปใช้ผู้ใช้งานได้เลือก ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลเส้นทางและปลายทาง

```

ResultSet myresult = stmt.executeQuery("SELECT * FROM matching WHERE word like
'%" + reqsearch + "%'");           //set sql command to search anywhere matched keyword
while(myresult.next()){
    output = output + myresult.getString("word") + ',';    //concatenate result with original output
}
output = output + '$';
out.println(output);    //return to client

```

4.5.4 ไฟล์ Server_RouteJunction.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลแยกของถนนหรือเรียกว่า โหนด เพื่อใช้ในการค้นหาเส้นทาง หลังจากที่ได้เส้นทางหรือปลายทางแล้วระบบจะคิดต่อมายังไฟล์ โดยไฟล์จะทำหน้าที่ในการรับข้อมูลชื่อเส้นทางหรือปลายทางจากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลในฟิลด์ชื่อเส้นทางหรือปลายทางตรงกับข้อมูลที่รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของข้อมูลแยกหรือโหนดของเส้นทางหรือปลายทาง ส่งกลับไปที่ฝั่งไคลเอนต์เพื่อนำไปใช้งานต่อไป ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลสมาชิก

```

ResultSet myresult = stmt.executeQuery("SELECT * FROM matching WHERE word='"+reqjunc+"'");
while(myresult.next()){           //set sql command to get junction nearly places
    output = output + myresult.getString("inter1") + ',' + myresult.getString("inter2") + ',';
}                                //get junction and concatenate with the original output
output = output + '$';           //return to client
out.println(output);

```

4.5.5 ไฟล์ Server_RouteResponse.jsp

เป็นไฟล์สำหรับการส่งข้อมูลไปยังโปรแกรมค้นหาเส้นทาง ไฟล์จะทำหน้าที่ในการรับข้อมูล ชื่อต้นทางและปลายทาง ได้แก่ ต้นทาง1, ต้นทาง2, ปลายทาง1, ปลายทาง2 และ ปัจจัยการค้นหาเส้นทาง จากนั้นใช้จาวาบิน ในการติดต่อไปยังโปรแกรมค้นหาเส้นทาง เมื่อการค้นหาเสร็จสิ้นระบบจะได้เส้นทางที่ดีที่สุดส่งกลับไปยังฝั่งไคลเอนต์ ซึ่งเป็นเส้นทางที่ผู้ใช้สามารถเดินทางได้อย่างมีประสิทธิภาพ

4.5.6 ไฟล์ Server_AreaSearch.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลเขตพื้นที่รายงาน เพื่อใช้ในบริการ รายงานสภาพจราจรและ บริการแก้ไขข้อมูลสมาชิก ซึ่งต้องมีการค้นหารายชื่อเขตที่ต้องการ ไฟล์จะทำหน้าที่ในการรับข้อมูลคีย์เวิร์ดเข้ามา จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาในแถวข้อมูลที่มีตัวอักษรตรงกับคีย์เวิร์ด ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของข้อมูลรายการเขตส่งกลับไปที่ฝั่งไคลเอนต์เพื่อนำไปเลือกพื้นที่รายงานต่อไป ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลเขต

```

ResultSet myresult = stmt.executeQuery("SELECT distr_name FROM district WHERE distr_name like
"+"%"+reqarea+"%");           //set sql command to search area matched keyword
while(myresult.next()){
    output = output + myresult.getString("distr_name") + ',';    //concatenate areas with the-
}                                                                //original output
output = output + "$";           //return to client

```

4.5.7 ไฟล์ Server_RangeSearch.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลพื้นที่ในเขตรายงาน จะทำงานหลังจากที่ผู้ใช้งานได้เลือกเขต แล้ว พื้นที่รายงานจะเป็นส่วนหนึ่งของเขตรายงาน เนื่องจากเขต 1 เขตมีขนาดใหญ่ โดยไฟล์จะทำหน้าที่ในการรับข้อมูลชื่อเขตรายงาน จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลในฟิลด์ชื่อเขตตรงกับข้อมูลที่ได้รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของรายการพื้นที่ต่างๆ ในเขตที่การ

รายงานสภาพจราจร และจะส่งกลับไปที่ฝั่งไคลเอนต์ให้ผู้ใช้ได้เลือกต่อไป ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลรายการพื้นที่

```
ResultSet myresult = stmt.executeQuery("SELECT * FROM report WHERE district='"+reqarea+"'");
while(myresult.next()){
    // set sql command to search area
    output = output + myresult.getString("street") + ','; //get the ranges in the area
}
output = output + "$"; //return to client
```

4.5.8 ไฟล์ Server_ReportResponse.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลรายละเอียดสภาพการจราจรในเขตพื้นที่ที่ได้เลือกไว้ ไฟล์จะทำหน้าที่ในการรับข้อมูลชื่อพื้นที่รายงานเข้ามา จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลในฟิลด์ชื่อพื้นที่ตรงกับข้อมูลที่ได้รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของข้อมูลรายละเอียดการรายงานสภาพการจราจรในพื้นที่บริเวณนั้น เช่น เกิดอุบัติเหตุ หรือ มีสิ่งก่อสร้าง เป็นต้น ข้อมูลจะถูกส่งกลับไปที่ฝั่งไคลเอนต์เพื่อนำไปแสดงผลออกทางจอภาพ และถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลรายละเอียดการรายงานสภาพการจราจร

```
ResultSet myresult = stmt.executeQuery("SELECT * FROM report WHERE street='"+reqstr+"'");
while(myresult.next()){
    // set sql command to get report by range in the area
    output = myresult.getString("date")+','+myresult.getString("comment");
}
//get report
output = output + "$"; //return to client
```

4.5.9 ไฟล์ Server_EditUpdate.jsp

เป็นไฟล์สำหรับการแก้ไขข้อมูลในฐานข้อมูลของสมาชิก โดยไฟล์จะทำหน้าที่ในการรับข้อมูลทั้งหมดของสมาชิก จากนั้นใช้คำสั่งอัปเดตฐานข้อมูล อัปเดตแถวของข้อมูลที่มีข้อมูลในฟิลด์ชื่อสมาชิกตรงกับข้อมูลที่ได้รับเข้ามา ถ้าการอัปเดตสำเร็จ ผลลัพธ์ที่ได้จะเป็นเลข 1 ถ้าค้นหาไม่พบผลลัพธ์จะเป็นเลขอื่นที่ไม่ใช่เลข 1 และจะส่งกลับไปที่ฝั่งไคลเอนต์เพื่อใช้งานต่อไป

```
String sql = "UPDATE member SET
usr='"+requser+"',pass='"+reqpass+"',name='"+reqname+"',email='"+reqemail+"',district='"+reqdistr+"',
address='"+reqaddr+"',mobile='"+reqmob+"',provider='"+reqprov+"',mo_usr='"+reqmouser+"',
mo_pass='"+reqmopass+"' WHERE usr='"+requser+"'"; //set sql to go update member information
int result = stmt.executeUpdate(sql);
```

```
if(result == 1)                                //update succeeded
    out.println("ok");
else                                            //update not succeeded
    out.println("no");
```